

Predicting Traffic Accident Severity

USING US ACCIDENTS DATASET

Big Data and Databases Group Project



Outline

01

Introduction
Overview

02

Data
Description

03

Data
Preparation

04

Modeling

05

Outcomes &
Comparisons

06

Managerial
implications

01

Introduction and Overview

Introduction

OVERVIEW OF THE ANALYSIS

- The actual database used in this project contains records of traffic accidents in the United States, with detailed information on geolocation, time, weather conditions, and road infrastructure.
 - In total, the dataset includes 500,000 samples and 46 variables. This is a reduced version chosen for computational feasibility. The goal of the study is to predict the severity of an accident and identify the key factors that most influence the likelihood of a serious incident. The target variable is represented as a binary indicator - Severity_bin.
 - From a management perspective, understanding the determinants of serious accidents allows authorities and transport services to optimize traffic safety measures, prioritize infrastructure investments, and improve emergency response. After describing and preparing the data, including handling missing values, forming weather conditions and time groups, removing redundant variables (which correlate with the basic ones), and balancing classes, Random Forest and H2O Gradient Boosting models were implemented for the classification task.
- 

Overview

OVERVIEW AND PHILOSOPHY BEHIND THE ANALYSIS

- We decided to compare multiple models for this analysis. We started with a Random Forest model with 100 trees, in 3 different variations. The original Random Forest showed good results and an acceptable recall, but was lackluster in precision and feature interpretability, while strongly correlating to others (like longitude and latitude). After modifying the original in two different versions, one where we decided to exclude accident coordinates, the other where balancing was applied. Results varied somewhat, for example when balancing precipitations became a major factor and without coordinates more economically interpretable features gained importance (crossings, roundabouts, etc.). Regarding AUC and recall though, the best model was the simplest, the original random forest. This brought us to the next step-
- Implementing Gradient Boosting where the focus shifted towards model optimization compared to data cleaning and feature selection. In the boosting model features were selected with the help of loops and the overall results were marginally superior to Random Forest. A pattern was emerging though, state seemed to be the most dominant feature in correlation. This leads us to our final step-
- A Multi-Layer Perceptron model where we played to its strength and simplified the dataset to exclude all but two states and compare the two. The results were unfortunately worse in regards to recall and AUC but nonetheless it served as valuable information in regards to managerial implications, demonstrating that state response is the most significant yet still relatively small part of what comprises the severity of an accident. Not everything can be managerially accounted for and controlled.

02

Data Description

Data Description - Univariate Analysis

► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table ◀ Flow Variables

Rows: 13 | Columns: 16

Table  Statistics 

☐	#	RowID	Column  String	Min  Number (...)	Max  Number (...)	Mean  Number (...)	Std. devia...  Number (...)	Variance  Number (...)	Skewness  Number (...)	Kurtosis  Number (...)	Overall su...  Number (...)
☐	1	Severit	Severity	1	4	2.213	0.487	0.237	1.811	3.269	1,106,374
☐	2	Start_L	Start_Lat	24.562	49	36.206	5.071	25.719	-0.074	-0.526	18,103,210.466
☐	3	Start_L	Start_Lng	-124.497	-67.484	-94.737	17.406	302.961	-0.479	-1.369	-47,368,291.523
☐	4	End_La	End_Lat	24.57	48.999	36.273	5.265	27.724	-0.159	-0.549	10,142,818.868
☐	5	End_Lr	End_Lng	-124.497	-67.484	-95.777	18.12	328.342	-0.36	-1.561	-26,781,327.115
☐	6	Distan	Distance(mi)	0	193.48	0.564	1.775	3.15	17.15	828.821	282,158.734
☐	7	Tempe	Temperature(F)	-77.8	207	61.646	19	361.005	-0.513	0.007	30,177,937.3
☐	8	Wind_C	Wind_Chill(F)	-53.2	207	58.229	22.352	499.623	-0.673	0.164	21,601,979.6
☐	9	Humidi	Humidity(%)	1	100	64.835	22.826	521.034	-0.396	-0.724	31,695,848
☐	10	Pressu	Pressure(in)	0.12	38.44	29.537	1.009	1.017	-3.742	19.408	14,504,607.52
☐	11	Visibili	Visibility(mi)	0	130	9.092	2.708	7.334	2.721	94.837	4,443,117.34
☐	12	Wind_S	Wind_Speed(mi)	0	822.8	7.681	5.431	29.5	8.44	1,111.406	3,556,563.3
☐	13	Precipi	Precipitation(in)	0	10.13	0.008	0.102	0.01	77.766	7,442.337	2,962.35

Made use of the Statistics Node for:

Summary statistics for central tendency
Measures of dispersion and variability

Data Description - Bivariate Analysis

OVERVIEW OF BIVARIATE ANALYSIS

- Finding Cramer's V for categorical variables
- Finding p-value and mean square using One Way Anova for continuous variables
- Significant findings include: State being weakly associated with severity (0.145 Cramer's V), precipitation being very strongly significant (p-value < 0.0001), visibility also being moderately significant (p = 0.011).
- Weather group and Crossing instead are both negligibly correlated with Cramer's V values below 0.1.
- Distance although logically presenting itself as a strongly correlated variable, actually presented a p-value leading to non-significance, this could be due to One-Way-Anova's extreme sensitivity to outliers but for conservation of proper data these outliers were not excluded

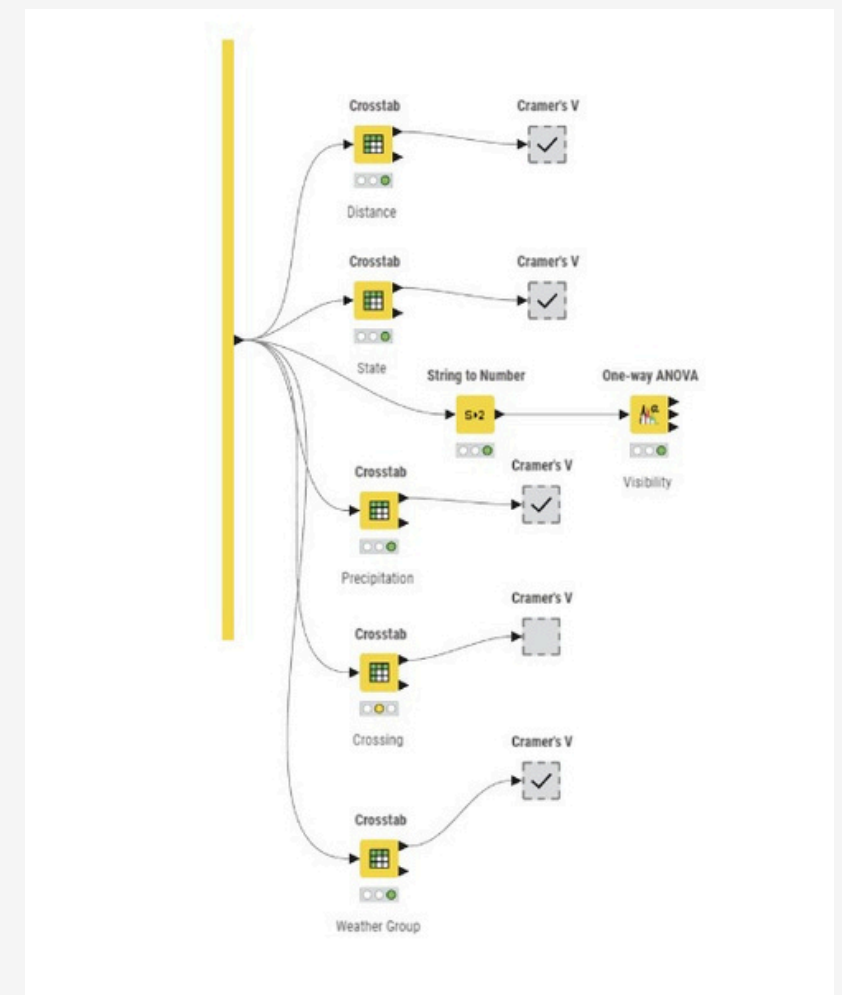


Table Statistics				
Row1	Sum(Row2)	Sum(Row11)	Min	Cramer'sV
	500,000	6,208.837	2	0.079
Sum of Squares	df	Mean Square	F	p-value
46.537	1	46.537	6.476	0.011
3,593,105.1	499998	7.186	?	?
3,593,151.637	499999	?	?	?

Top: "Crossing" Cramer's V
Bottom: Visibility One-Way Anova

03

Data Preparation

- Column selection
- Missing value & precipitation flag
- Weather condition grouping
- Target binarization
- Time feature engineering
- Rush hour & weekend indicators

Data Preparation Overview

Data Cleaning

At this stage, initial data cleaning was performed. Using Column Filter, service and text variables that do not contain useful information for predicting the severity of accidents (text descriptions, IDs, address fields, etc.) were removed, and in the Mission Value knife, missing values in weather variables (temperature, wind speed, humidity, pressure, visibility, precipitation, etc.) were replaced with the median for the column.

Data Preparation Weather & Target

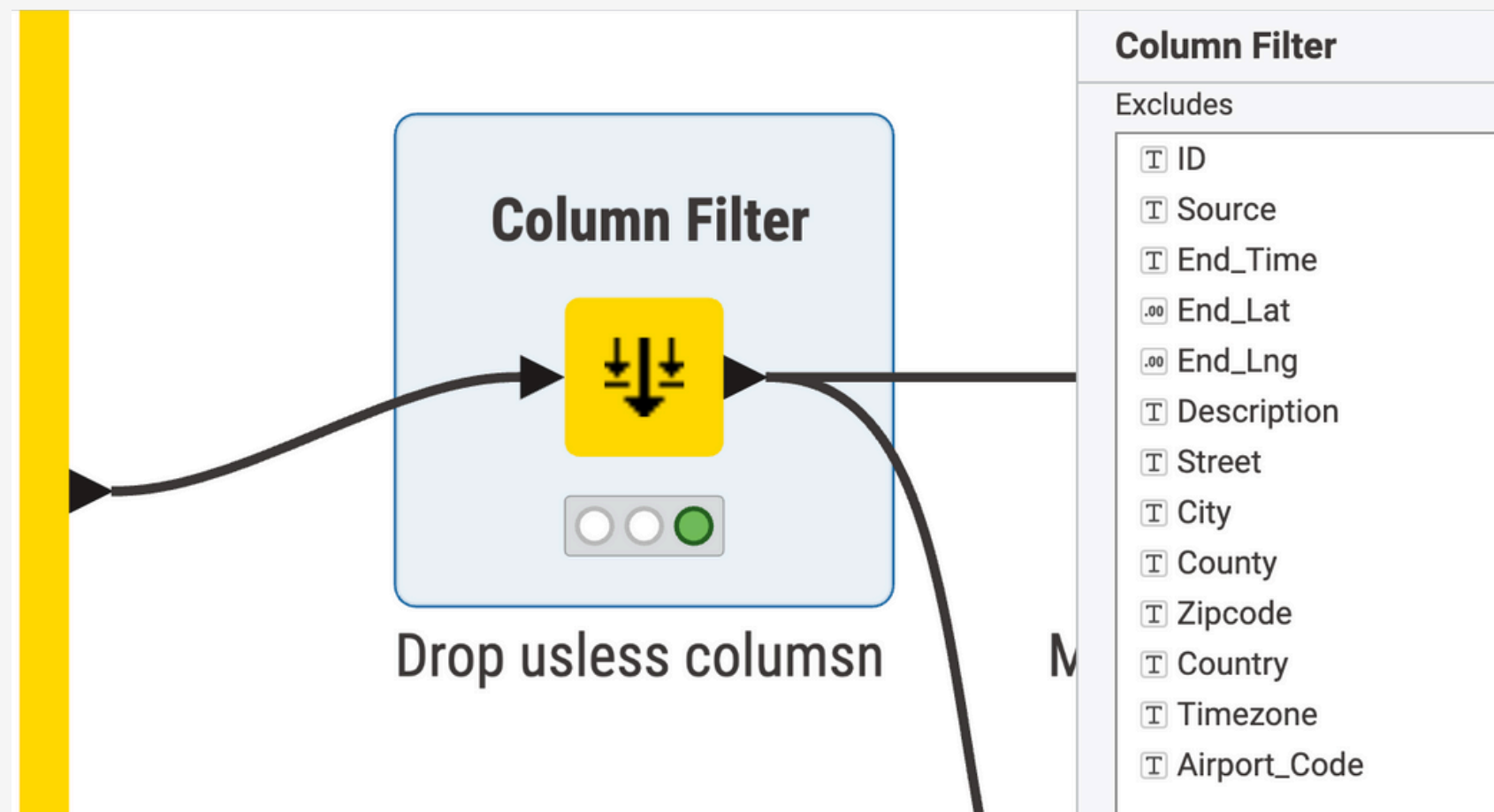
In this part, the main feature transformations were performed. Using Rule Engine, the Weather_Group variable was created, which aggregates detailed text descriptions of the weather into several understandable categories (CLEAR, CLOUDY, RAIN, SNOW, FOG, etc.). At the same time, the target variable Severity_bin was formed: this indicator reflects not the medical consequences of road accidents, which are absent in the dataset, but is determined by the degree of their impact on road traffic, where values 3-4 are classified as serious accidents.

Data Preparation Time Features

The final stage of data preparation involves transforming the Start_Time variable, converting it to datetime format, and extracting the hour, day of the week, and month. Additionally, Rush_Hour and Weekend flags are generated to reflect the main time patterns of traffic accidents.

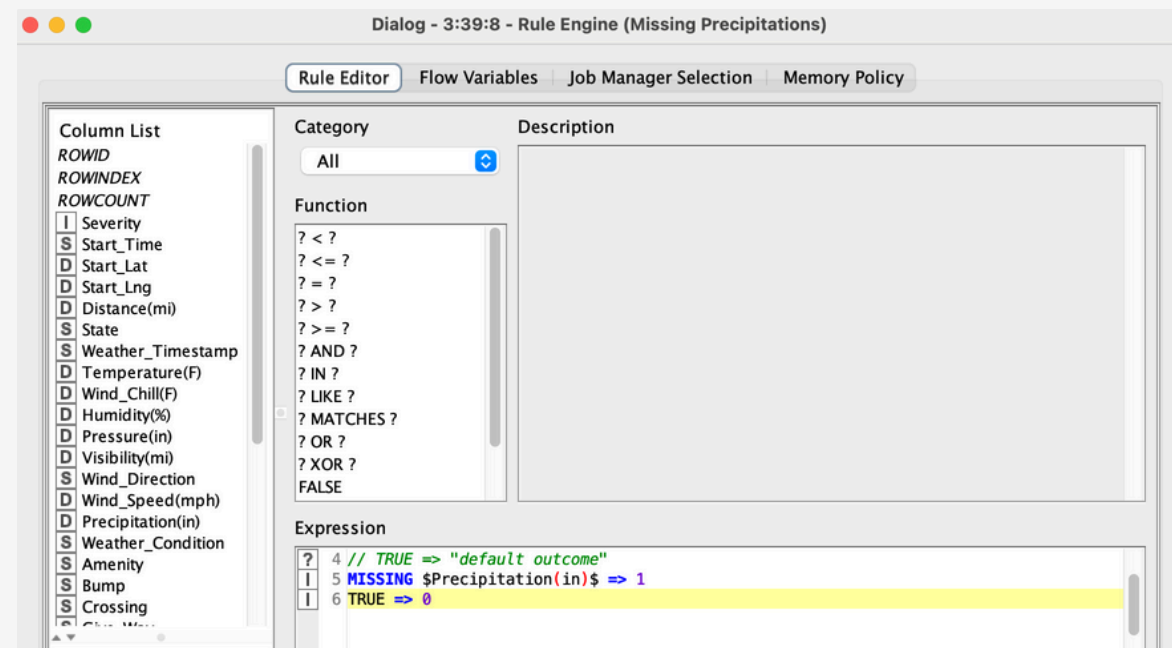
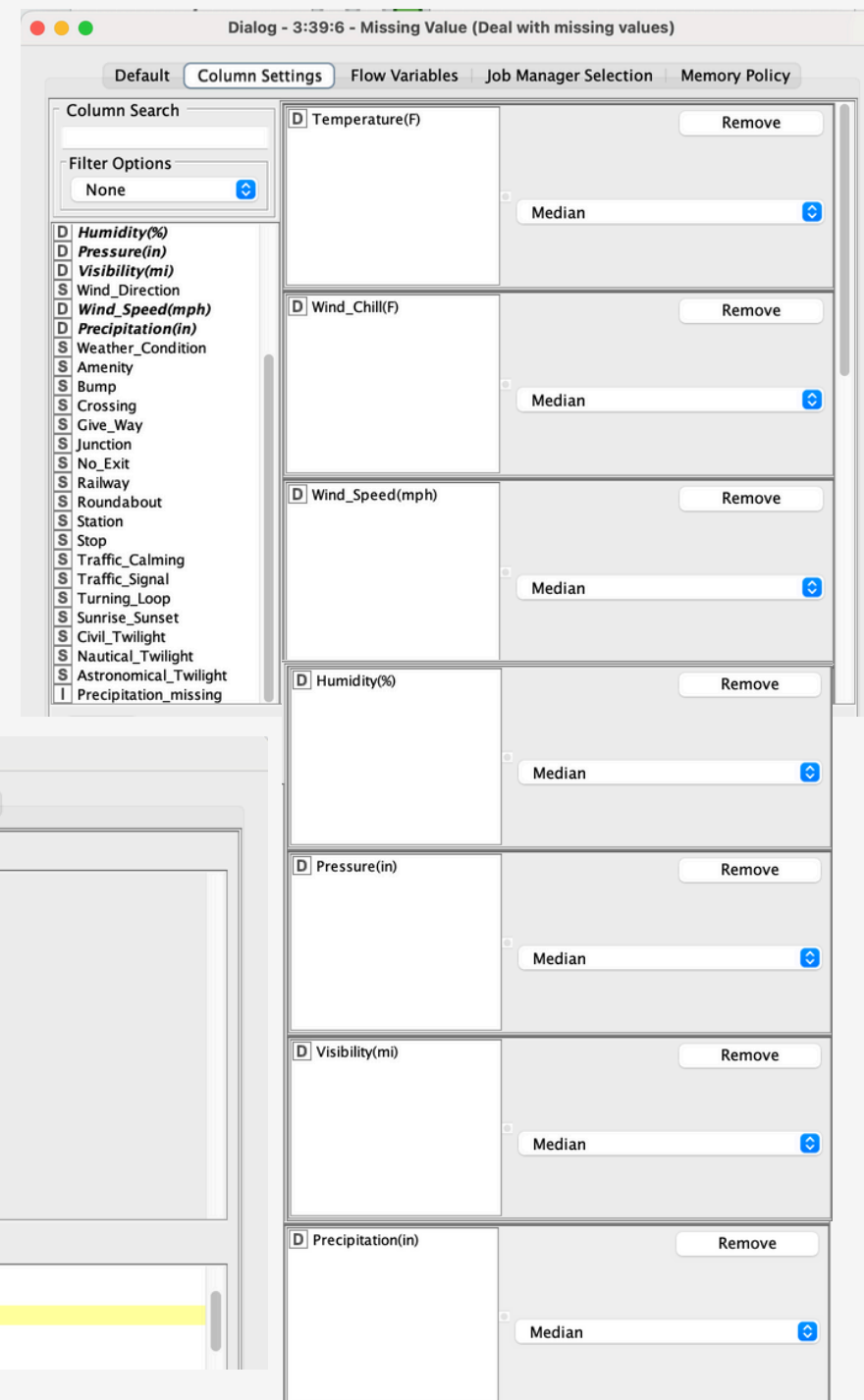
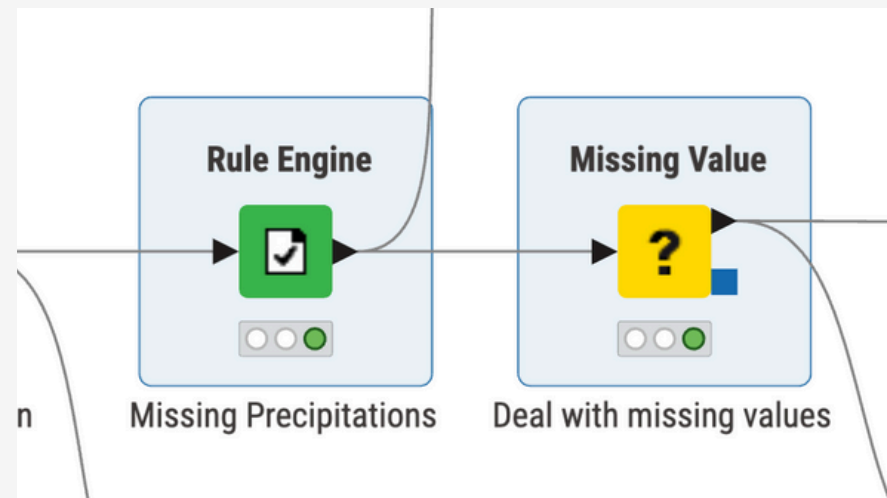
Data Cleaning

Column Selection



The initial dataset contained many service text and duplicate variables that had no modeling value or duplicated information about the accident location. To eliminate redundant attributes, a node Column Filter was applied to remove identifiers (ID and Source), address fields (Street, City, Country, Zipcode), duplicate coordinates (End_Lat and End_Lng), and other text descriptions of accidents. This step reduced the size of the dataset and allowed us to focus on variables relevant to predicting accident severity.

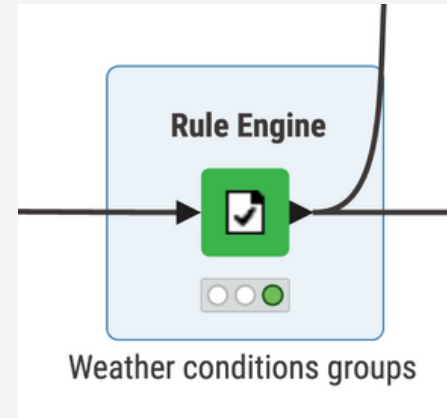
Data Cleaning



Missing value & precipitation flag

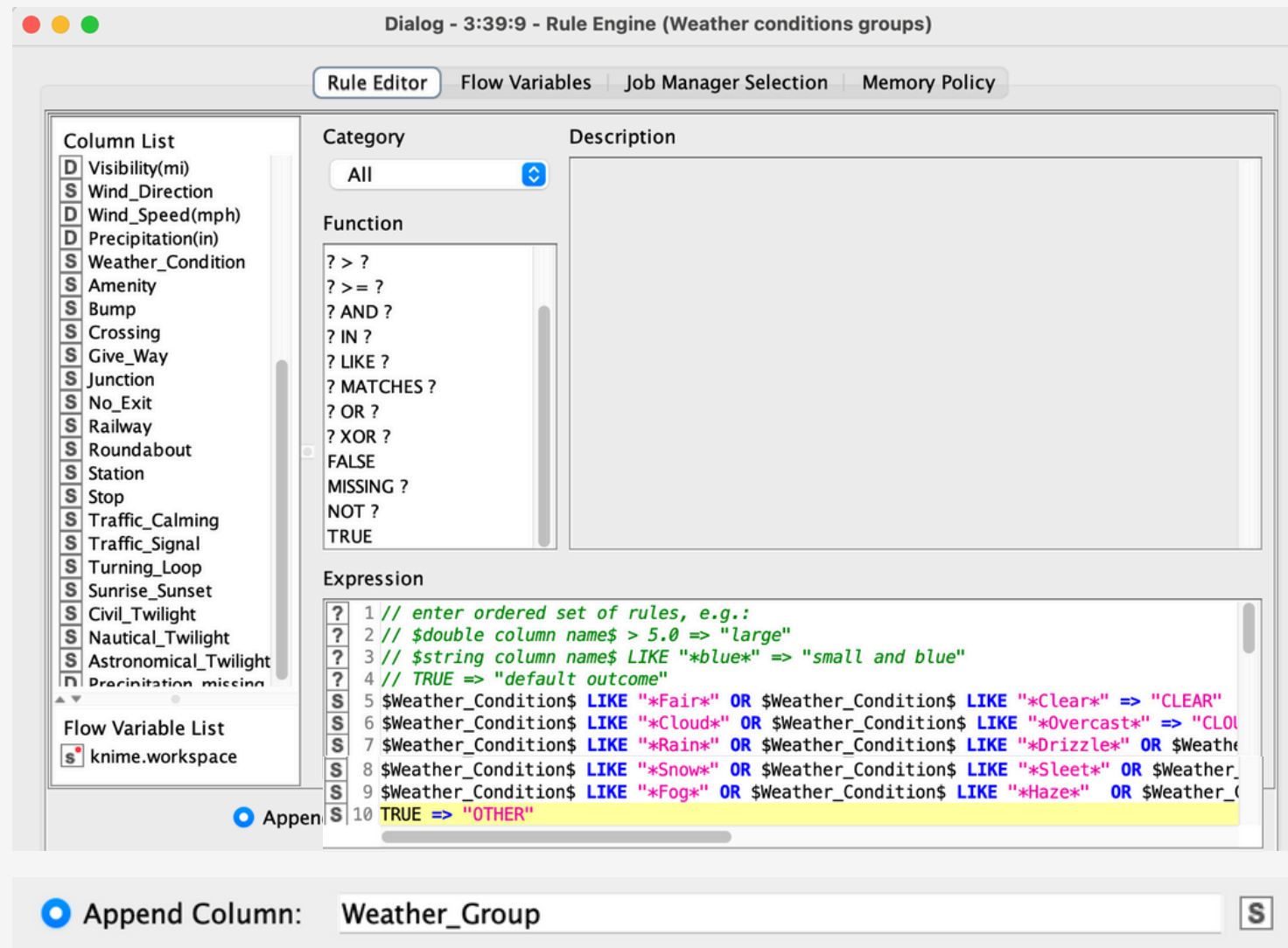
To account for missing values in the Precipitation(in) variable, an additional indicator Precipitation_missing was created using the Rule Engine node, which records cases of missing data. Further processing of missing values was performed in the Missing Values node, where gaps in key weather variables were replaced with corresponding median values. This approach allows us to obtain complete, consistent weather indicators and minimize the impact of missing values on the models.

Data Preparation Weather & Target

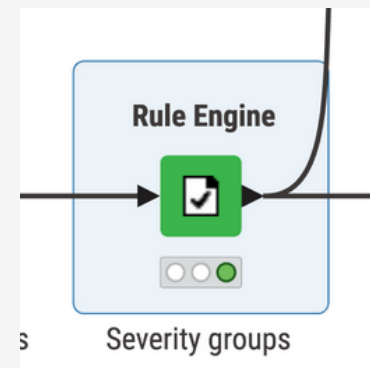


Weather condition grouping

The initial Weather_Condition change contained a large number of rare heterogeneous text values, which complicated modeling and interpretation. Using the Rule Engine node, these records were aggregated into several generalized categories: CLEAR, CLOUDY, RAIN, SNOW, FOG, and OTHER. This grouping reduces the number of unique factor levels, eliminates random, infrequent values, and converts weather information into a more consistent and analytically useful format. As a result, the Weather_Group variable became compact, stable, and fully suitable for use in modeling traffic accident severity classifications.



Data Preparation Weather & Target



Target Binarization

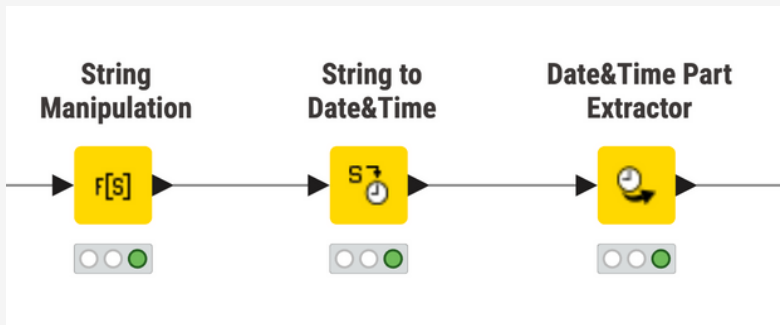
The Severity variable in the source dataset, represented by four levels, was converted into a binary indicator `Severity_bin` using the Rule Engine node. Values 3 and 4 are designated as serious accidents, while values 1 and 2 refer to non-serious cases. It is important to emphasize that the Severity variable in this dataset reflects not the medical consequences of an accident, but the degree of its impact on traffic. This logic allows the task to be formulated as a clear binary classification and ensures the consistency of subsequent forecasting models.

The screenshot shows the 'Rule Engine (Severity groups)' dialog box. The 'Rule Editor' tab is active. On the left, a 'Column List' contains various weather and traffic-related variables. The 'Function' list includes logical operators like '>', '>=', 'AND', 'IN', 'LIKE', 'MATCHES', 'OR', 'XOR', 'FALSE', 'MISSING', 'NOT', and 'TRUE'. The 'Expression' field contains a list of rules, with the fifth rule highlighted: `$Severity$ >= 3 => "1"`. The 'Append Column' section at the bottom shows 'Severity_bin' as the new column to be added, while 'Weather_Group' is listed as the source column.

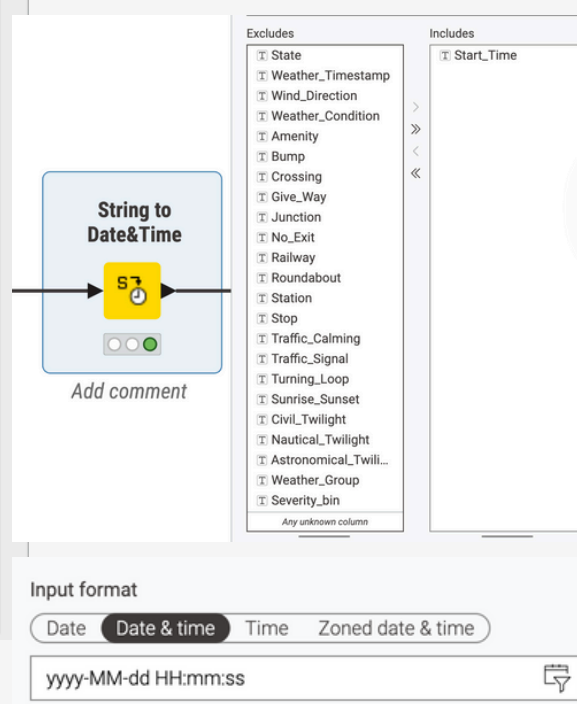
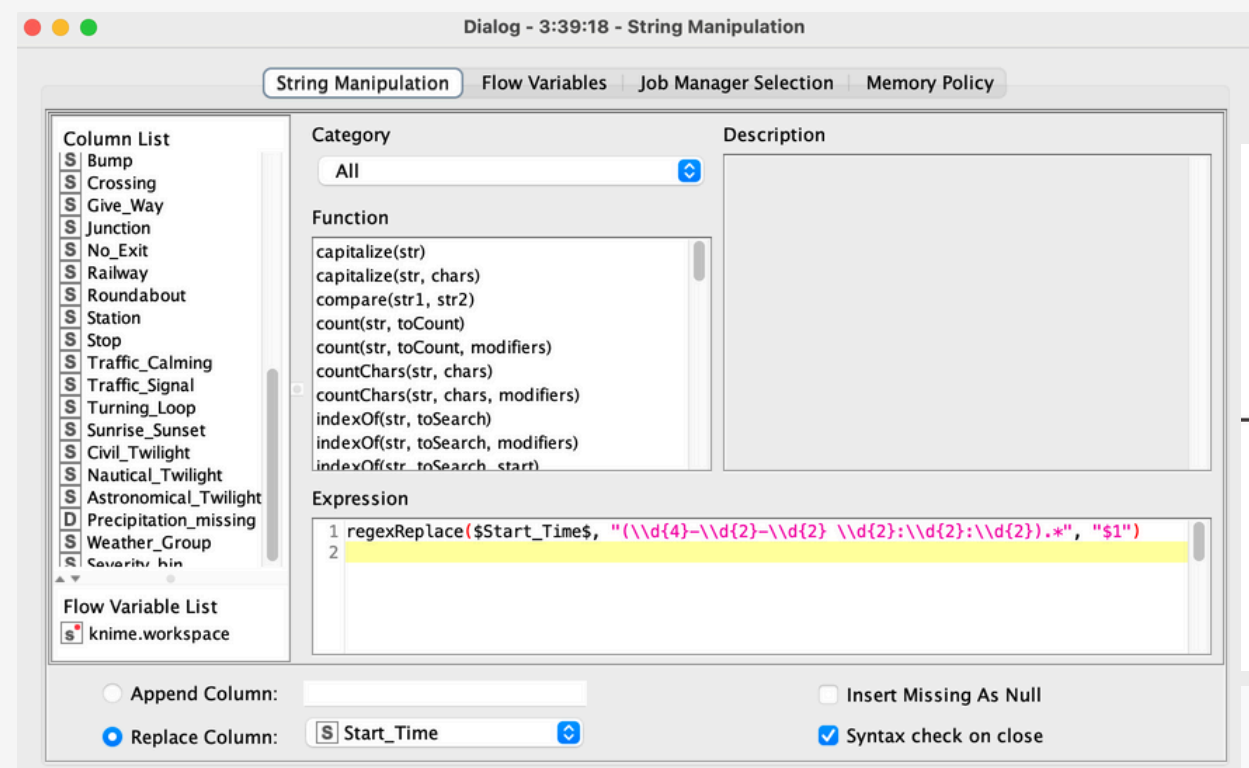
```
1 // enter ordered set of rules, e.g.:
2 // $double column name$ > 5.0 => "large"
3 // $string column name$ LIKE "*blue*" => "small and blue"
4 // TRUE => "default outcome"
5 $Severity$ >= 3 => "1"
6 TRUE => "0"
```

Data Preparation

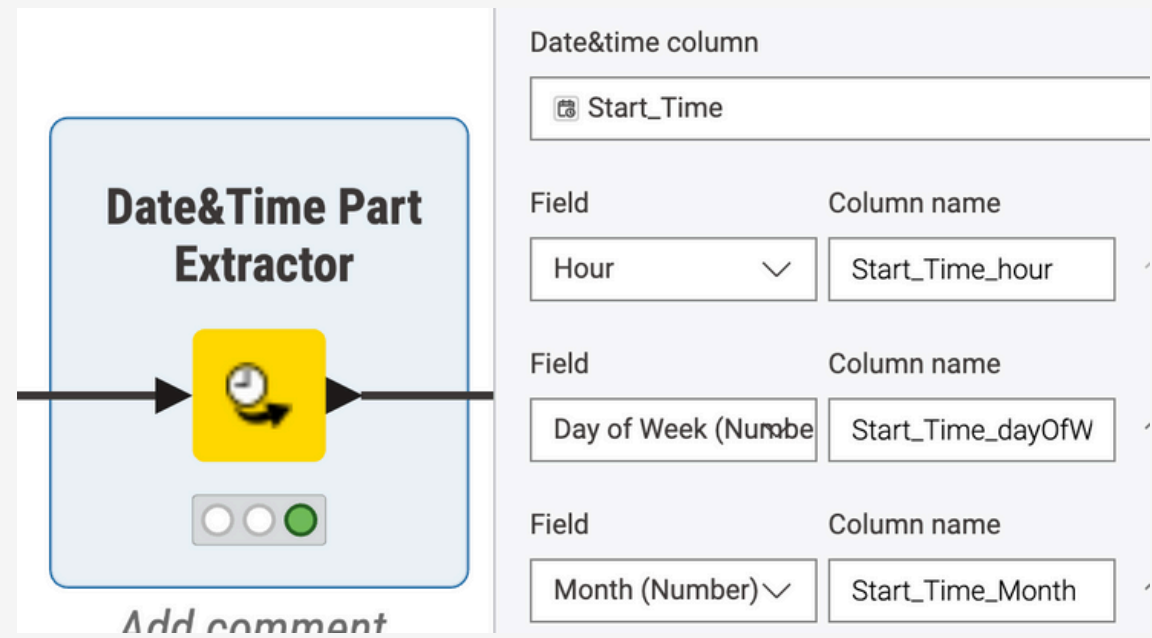
Time Features



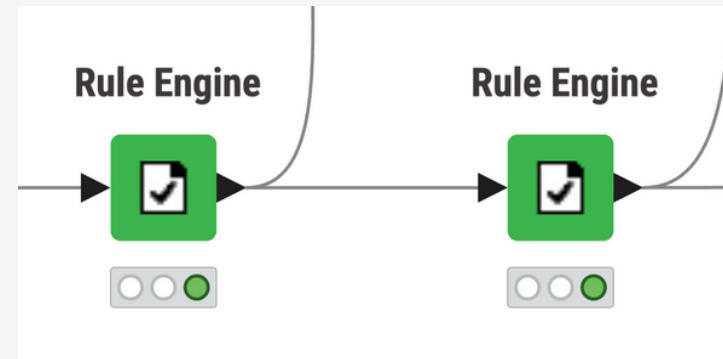
Time Features Engineering



The initial variable Start_Time contained the text format of the date and time, so before modeling, it was sequentially cleaned and structured. In the String Manipulation node, unnecessary elements were removed and the values were converted to the standard format "yyyy-MM-dd HH:mm:ss." Next, using String to Date&Time, the variable was converted to the datetime type. The final step was to extract key time components in the Date&Time Part Extractor node, which created the variables Start_Time_hour, Start_Time_dayOfWeek, and Start_Time_Month. This conversion allows the model to take into account daily and seasonal patterns of traffic accidents.

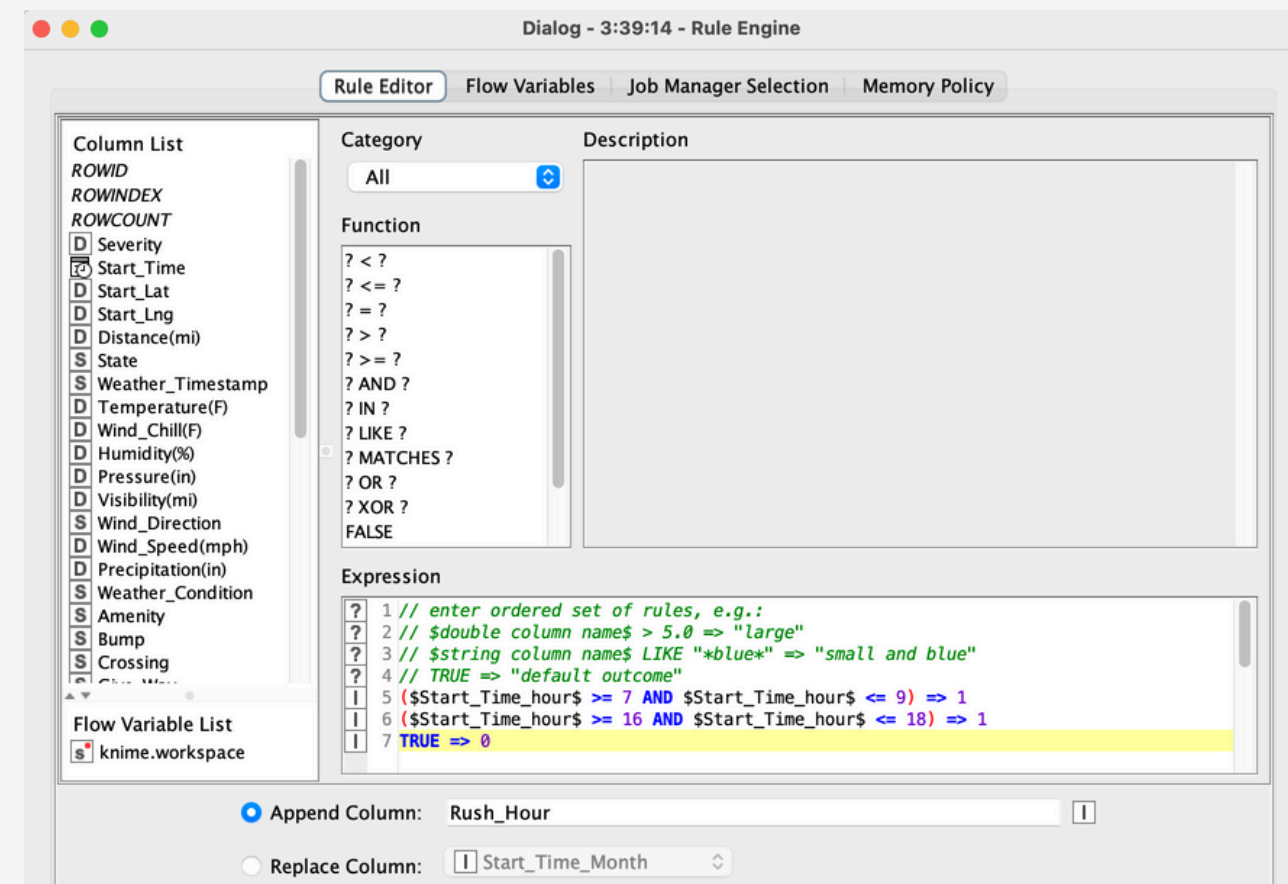
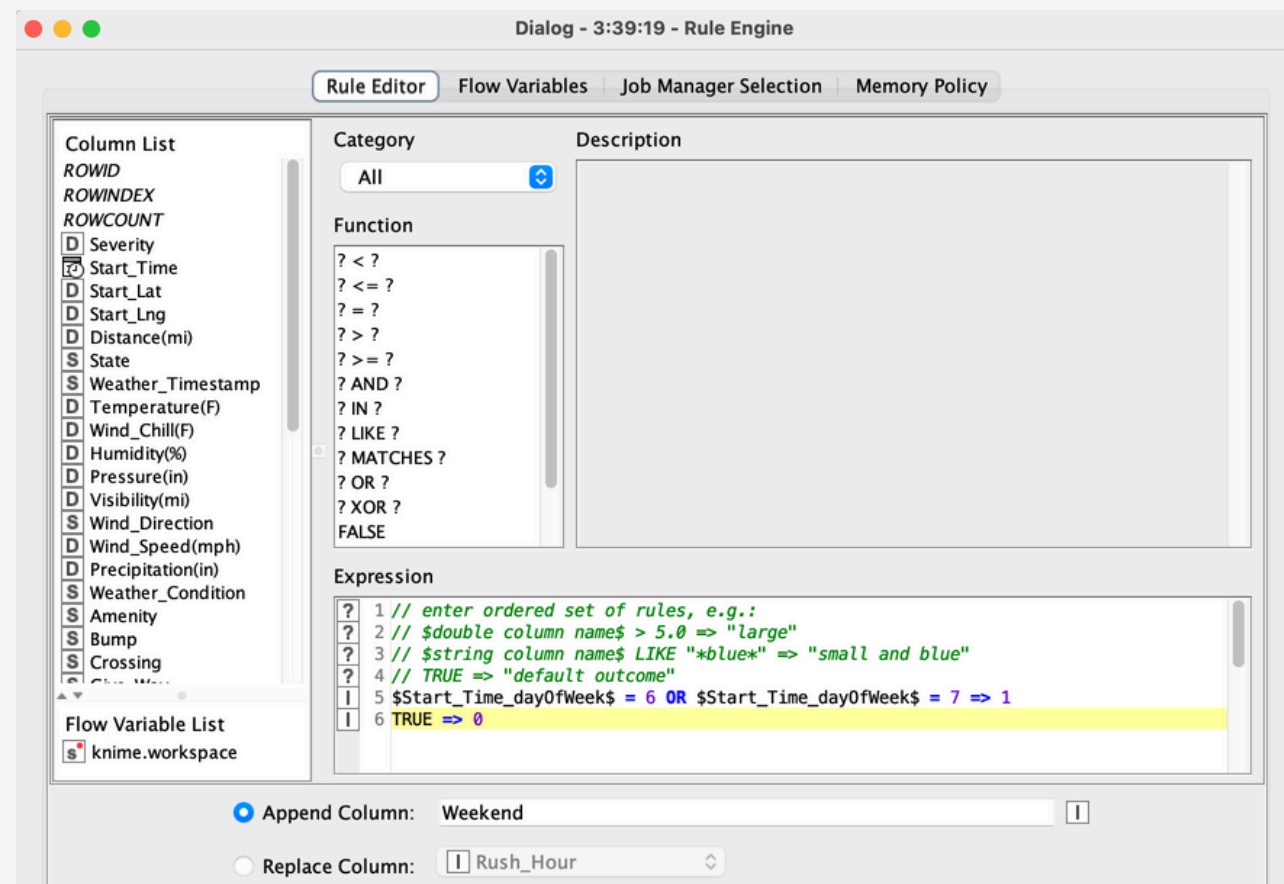


Data Preparation Time Features



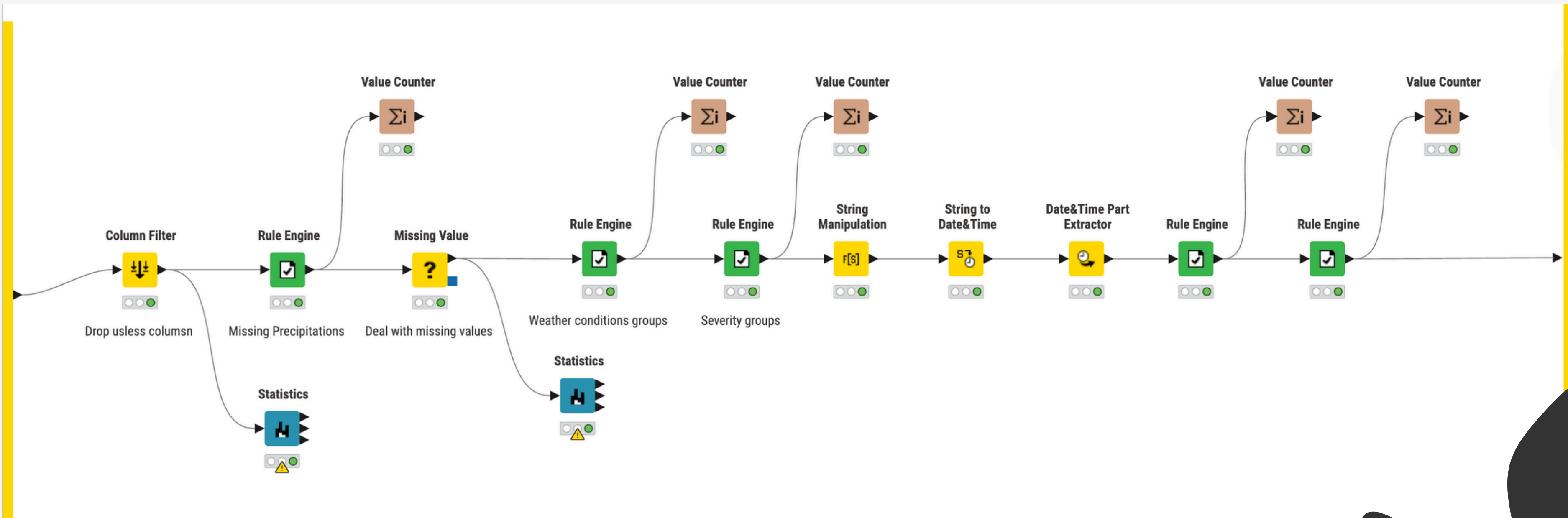
Rush hour & weekend indicators

To better account for temporal patterns of road activity, additional binary features were generated using the Rule Engine node. The Rush_Hour variable takes the value 1 during morning (7–9) and evening (16–18) peak intervals, reflecting increased traffic load. The Weekend variable denotes Saturday and Sunday. Both variables contribute to more accurate modeling of the impact of daily and weekly cycles on the probability of serious traffic accidents.



Data Preparation

The whole structure



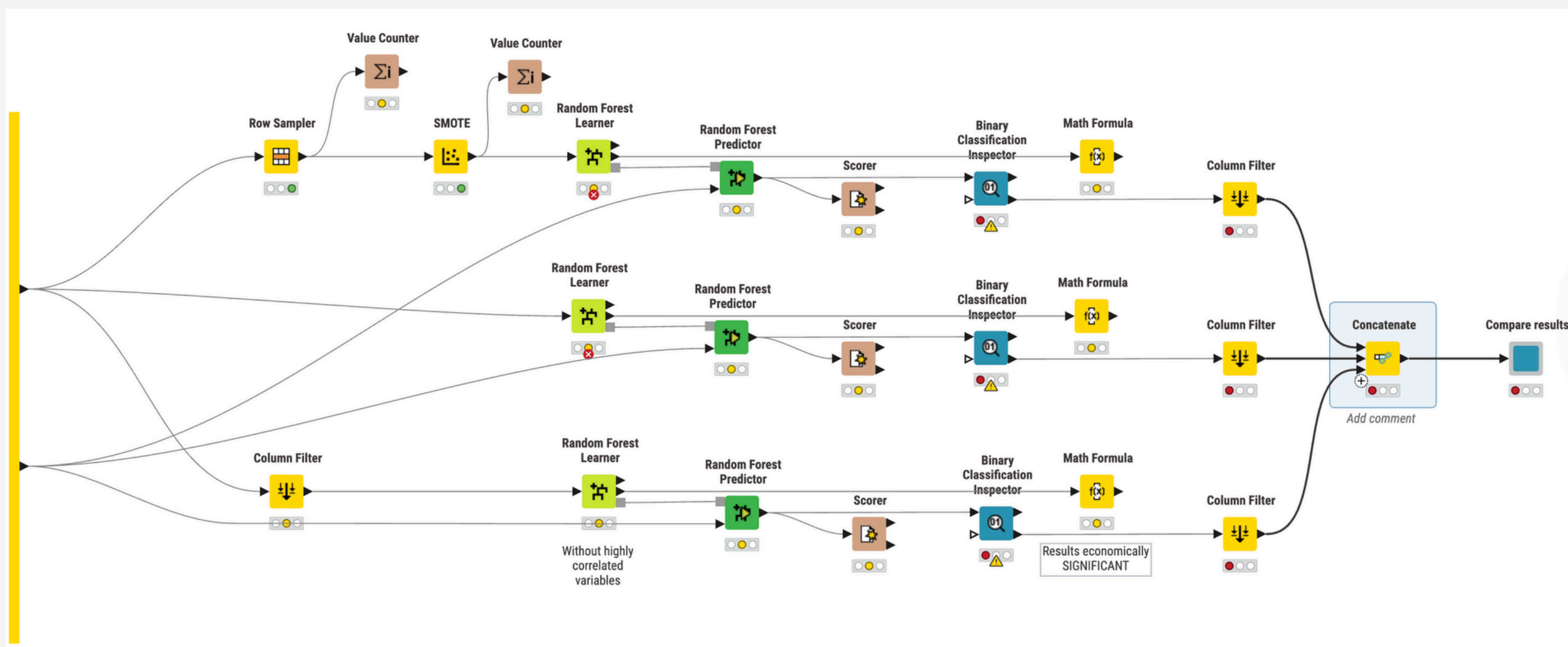
04

Modeling

- Random Forest
- H2O Gradient Boosting
- XGBoost

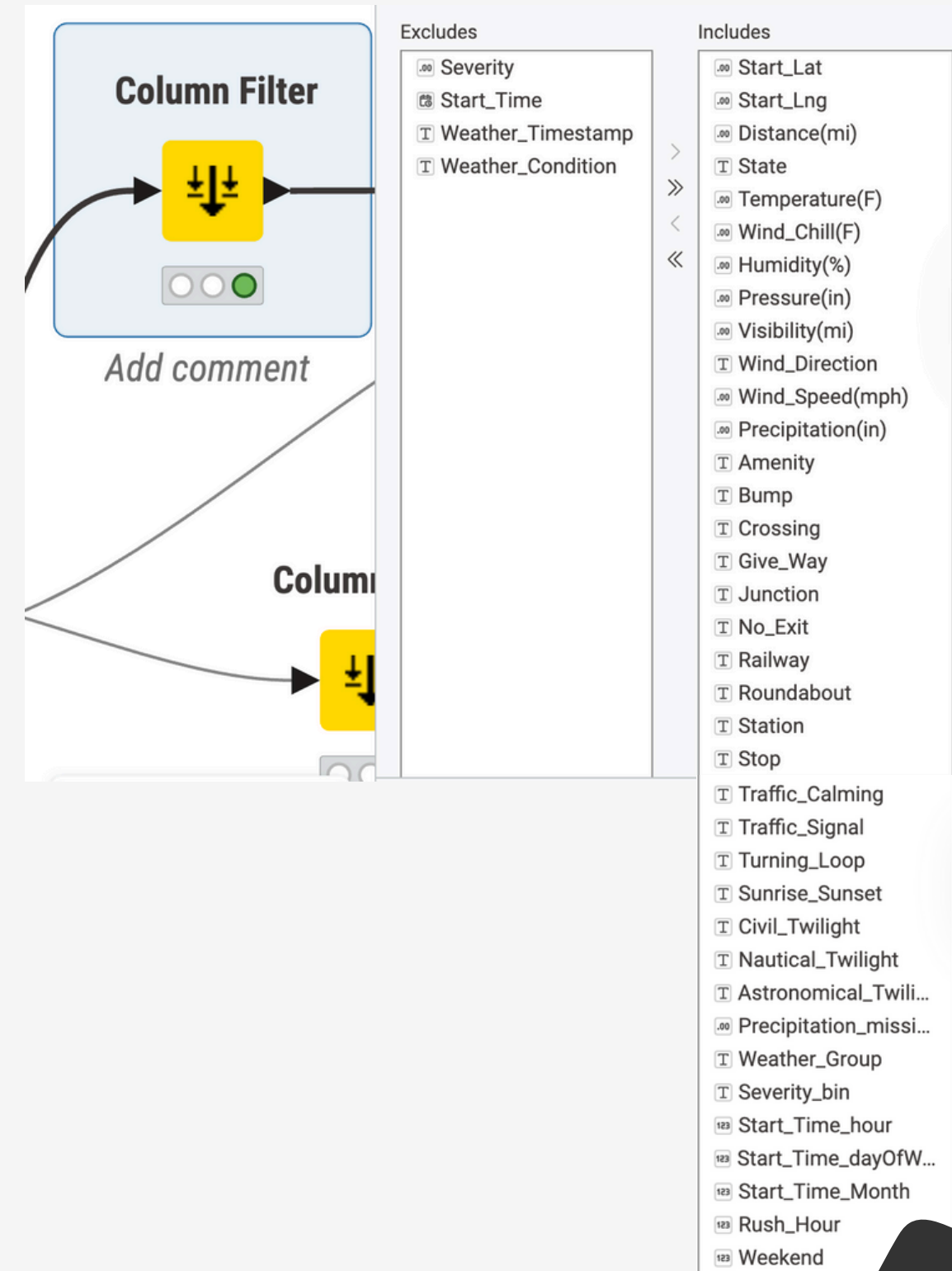
4.1 - Random Forest

- Balanced RF (SMOTE)
- Simple RF
- Reduced Variables RF

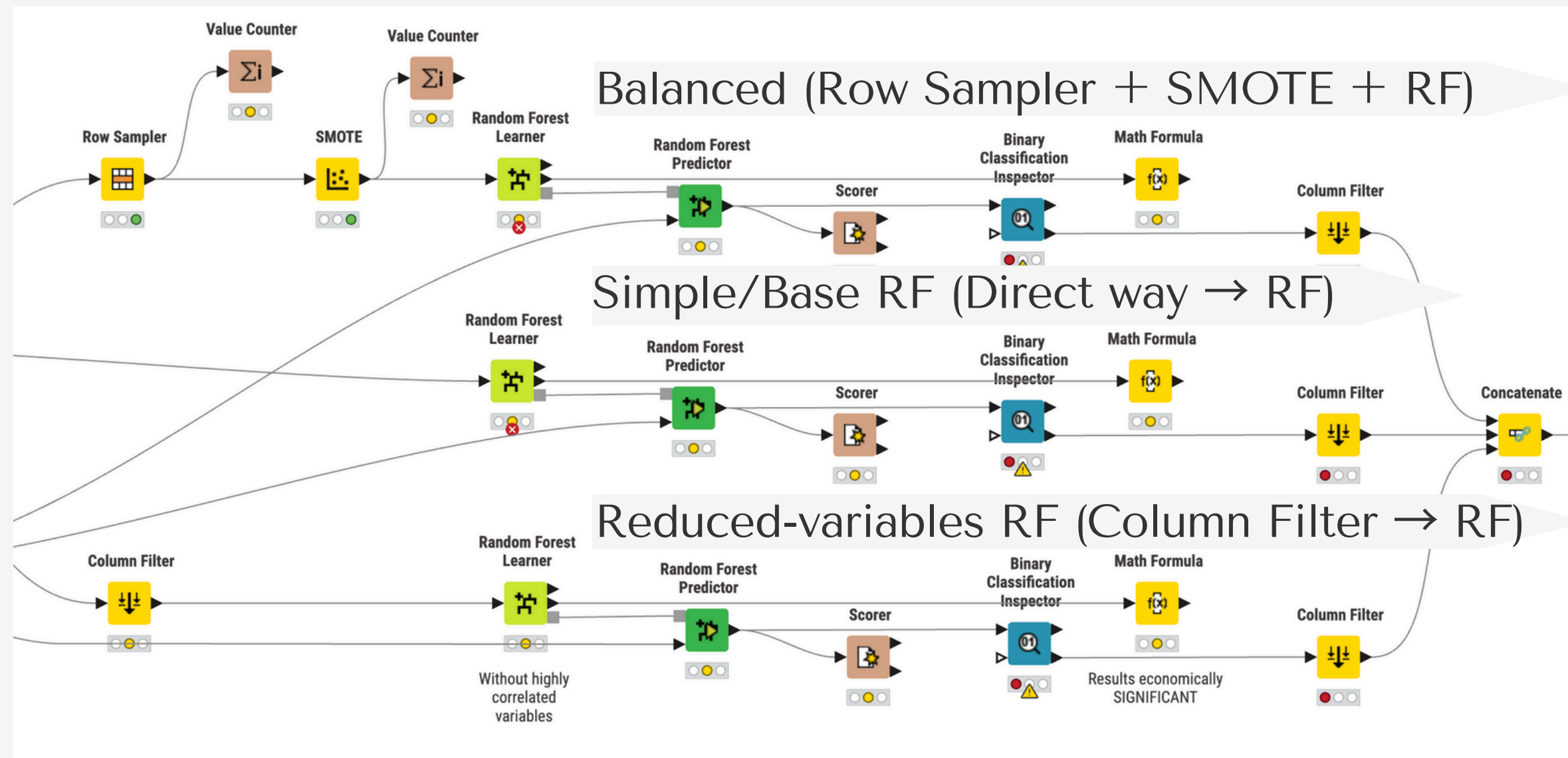


Additional Data Preparation for Random Forest

Before running Random Forest models, we perform a separate feature cleaning step using the Column Filter node. At this stage, variables that should not be included in the model are removed from the data: the target variable *Severity*, as well as time and text fields (*Start_Time*, *Weather_Timestamp*, *Weather_Condition*). All other spatial, weather, and infrastructure variables remain as potential predictors. This step ensures that the model is trained correctly: it receives only allowed predictors, does not contain explicit information about the target, and maintains the consistency of the input feature set for all three Random Forest configurations (balanced, simple, reduced). This also ensures the correctness of further comparison of models with each other.

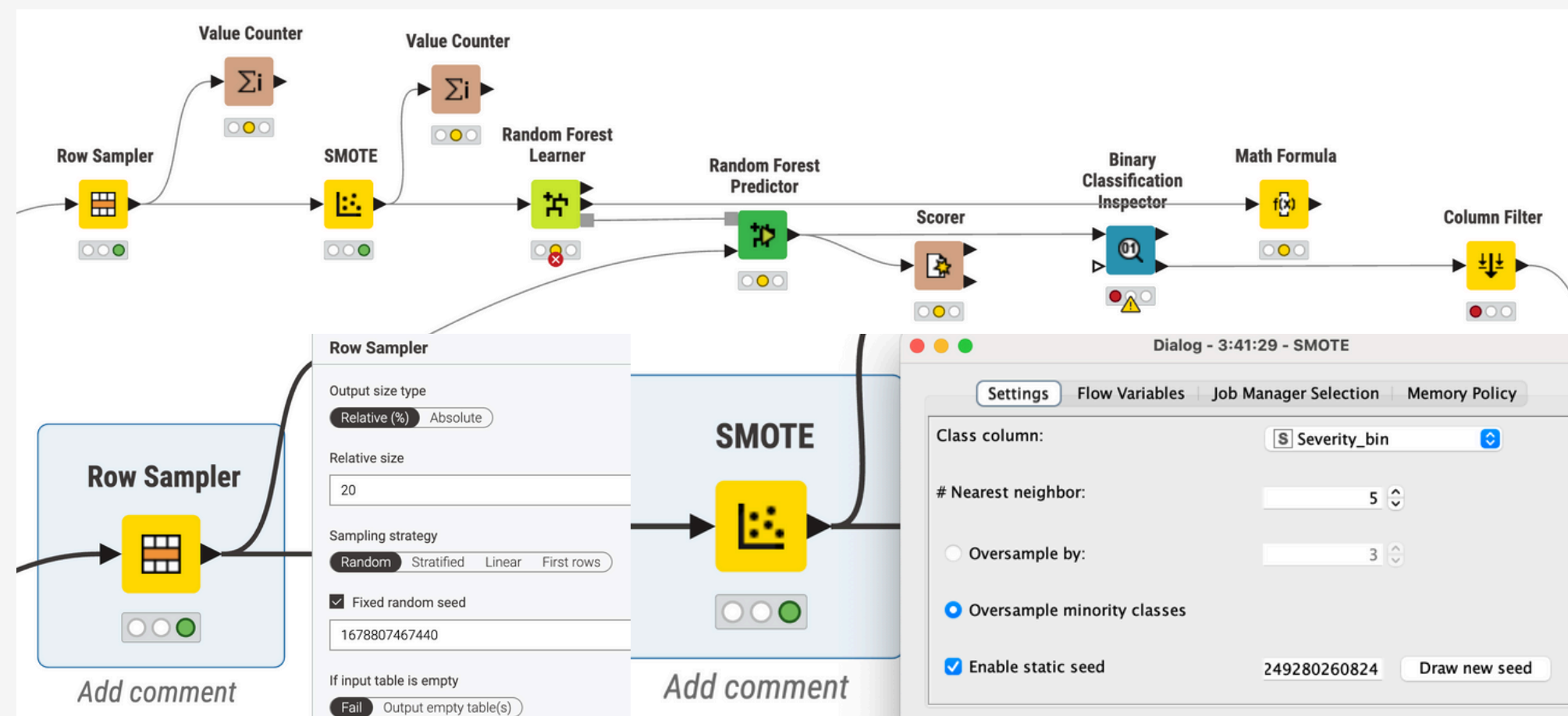


3 Branches of Random Forest



Random Forest modeling is divided into three parallel branches, which test separate scenarios for the algorithm. This approach allows us to assess how much the model depends on geographical and weather features and whether it can remain stable when the data structure is simplified.

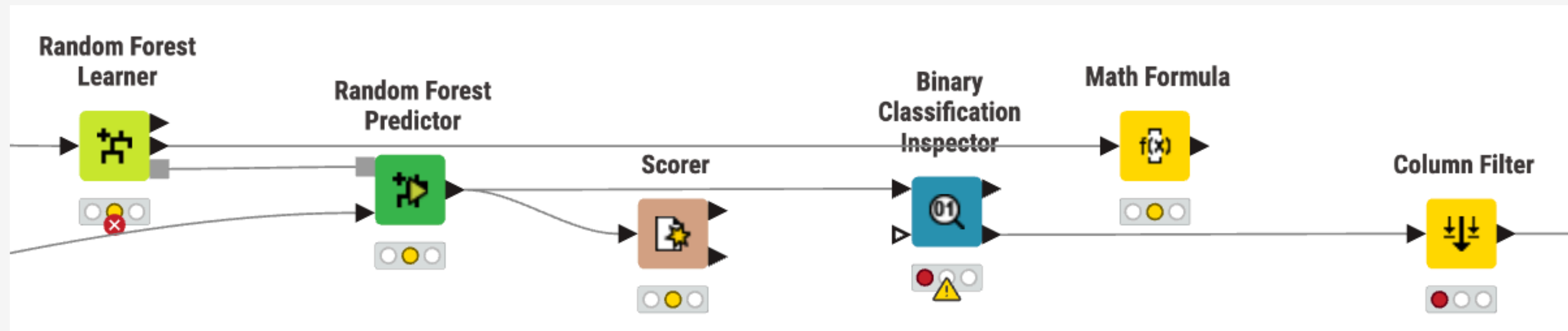
1) Balanced



The balanced branch uses Row Sampling and SMOTE, artificially increasing the proportion of severe cases. First, Row Sampler reduces the data volume to 20% to reduce the influence of the dominant class. Next, SMOTE generates synthetic examples for the minority class (Severity_bin = 1), increasing its representation in the data.

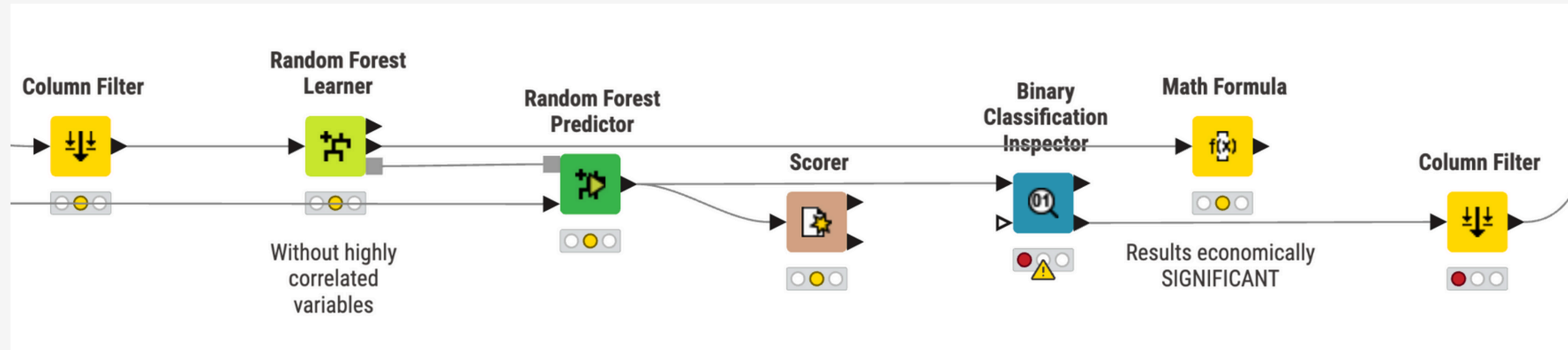
This makes it possible to evaluate how the model performs under balanced sampling conditions and whether its ability to detect rare events increases.

2) Simple

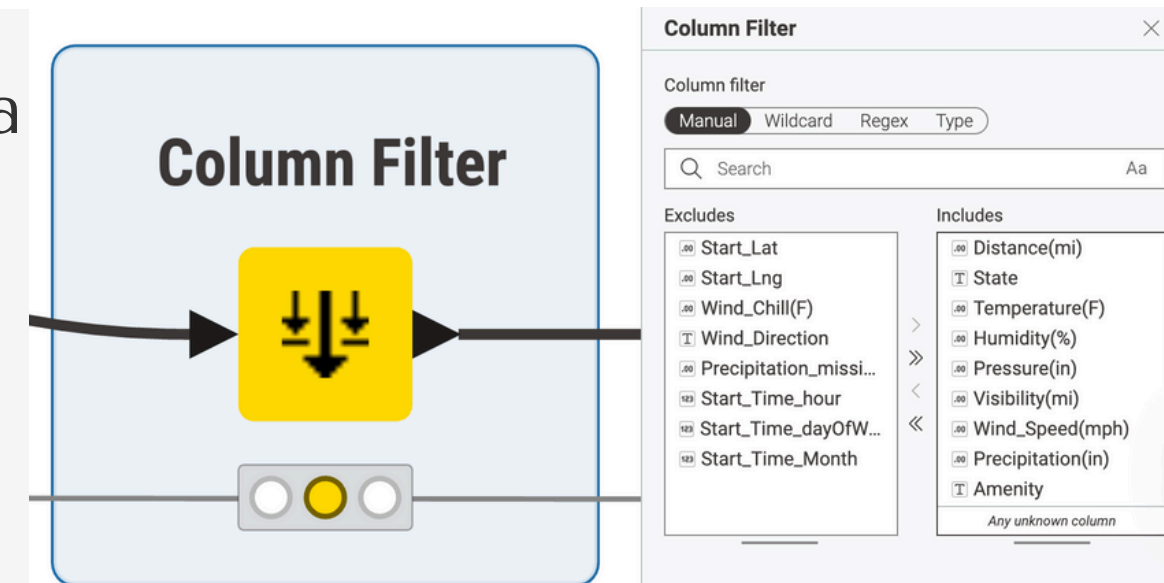


Simple Random Forest uses the full set of features obtained after Data Preparation, without any additional balancing or feature reduction. This allows the model to work with the actual distribution of severe/non-severe cases, showing its behavior in conditions that are as close to real life as possible.

3) Reduced Variables

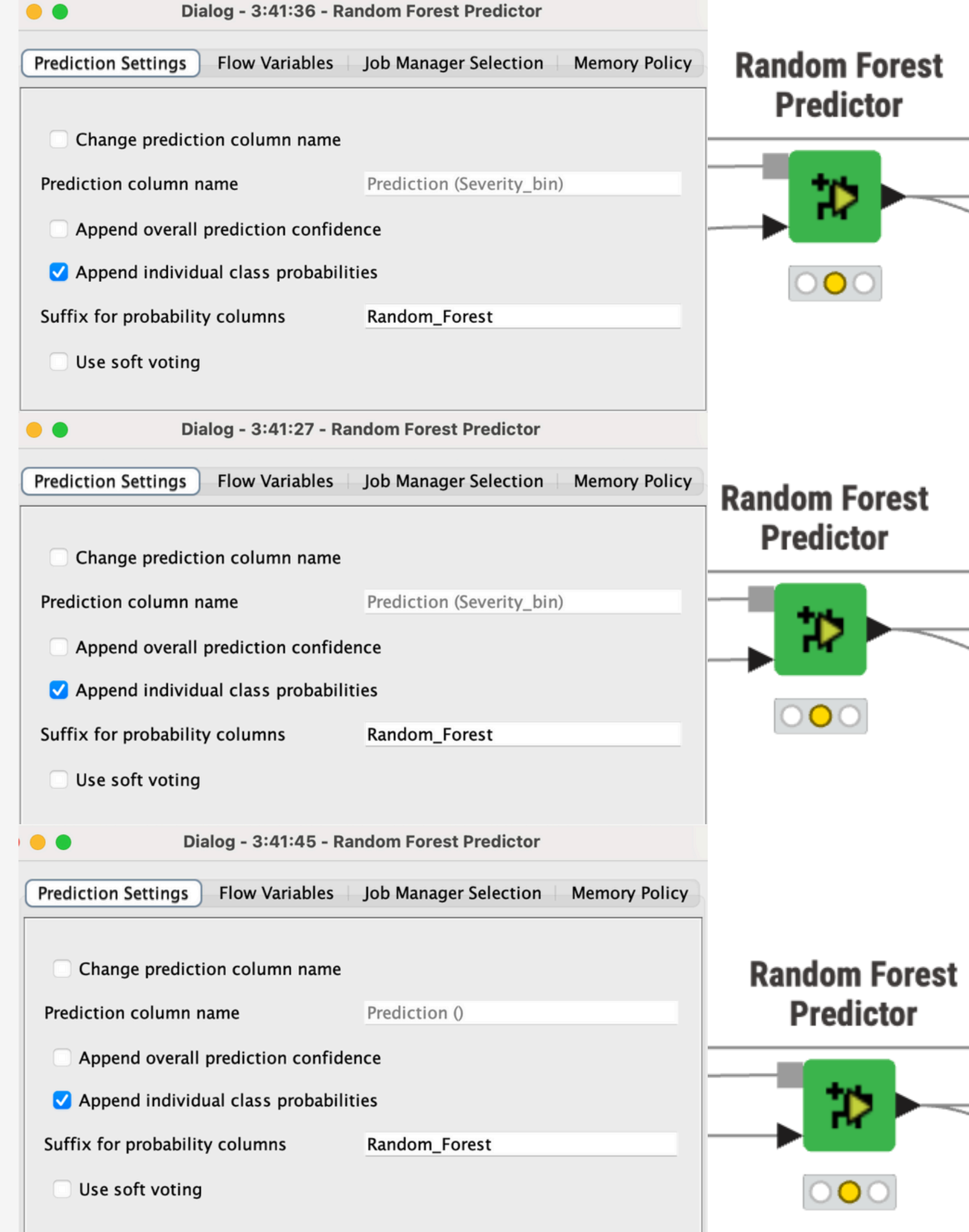


In the branch with reduced variables, the model is trained on a significantly simplified set of features. Variables that may be difficult to interpret or highly correlated with others, such as coordinates, weather parameters, and twilight indicators, are removed. This configuration allows us to investigate how important additional information is for the Random Forest model and which features have the most critical impact on the accuracy of predictions.



Random Forest Predictor

All three Random Forest Predictor nodes perform the same function: they apply the trained Random Forest model to the test data and generate two key outputs—a class prediction label (0/1) and the probability of belonging to the severe class. All models return individual class probabilities, which allows us to subsequently determine the optimal classification threshold in the Binary Classification Inspector. The predictor node parameters are intentionally the same in all three branches so that the models can be correctly compared with each other. The only difference is in the name of the output prediction column: the balanced and base models explicitly use Prediction (Severity_bin), while the reduced-features model creates a column by default. Due to the unified settings, the results of the three specifications are easily integrated into a single table for further performance analysis.



Random Forest Scorer

Scorer calculates the main classification metrics by comparing the actual Severity_bin values with the model predictions. It generates a confusion matrix and accuracy, sensitivity, specificity, and precision metrics, which are then used to summarize the results. The parameters are the same in all three models, which ensures a correct comparison between specifications. The consistent output format makes it easy to integrate metrics into subsequent nodes (Math Formula and Concatenate). Scorer does not change the data, but only evaluates the quality of the predictions.

Dialog - 3:41:28 - Scorer

Scorer | Flow Variables | Job Manager Selection | Memory Policy

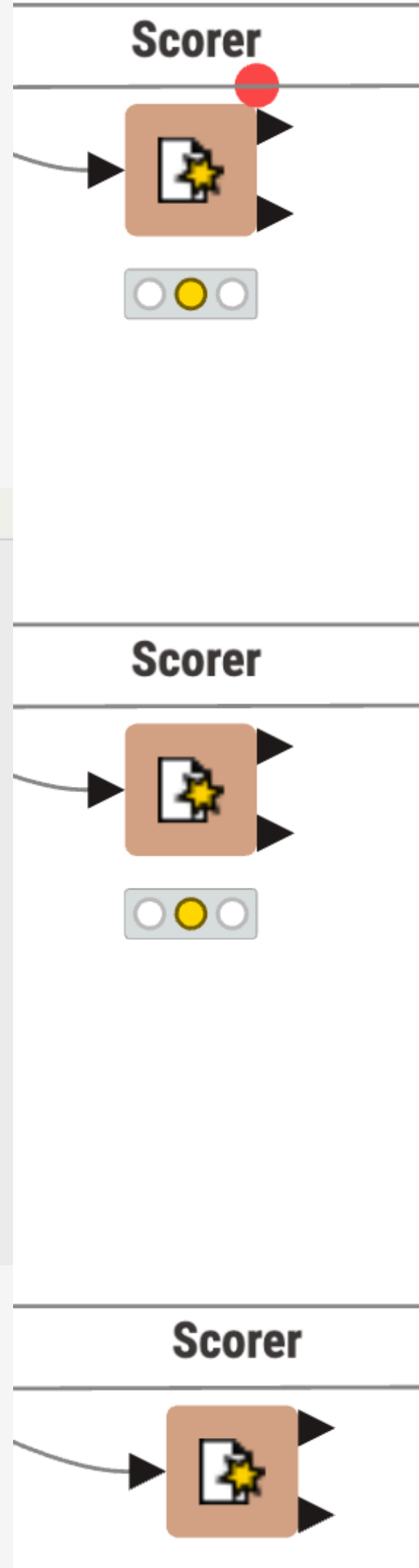
First Column
Severity_bin

Second Column
Prediction (Severity_bin)

Sorting of values in tables
Sorting strategy: Insertion order ☐ Reverse order

Provide scores as flow variables
☐ Use name prefix

Missing values
In case of missing values: ☒ Ignore ☐ Fail

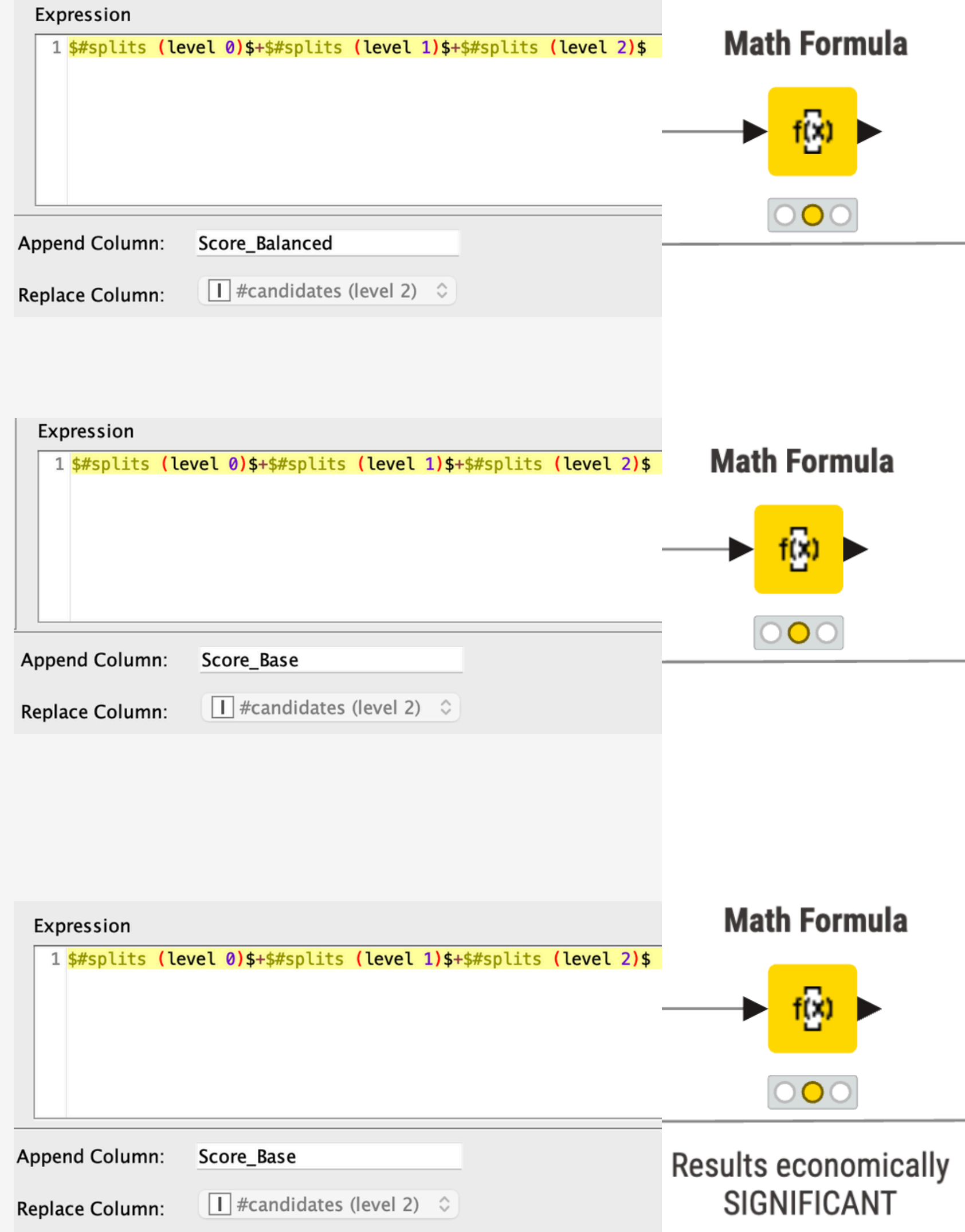


Math Formula

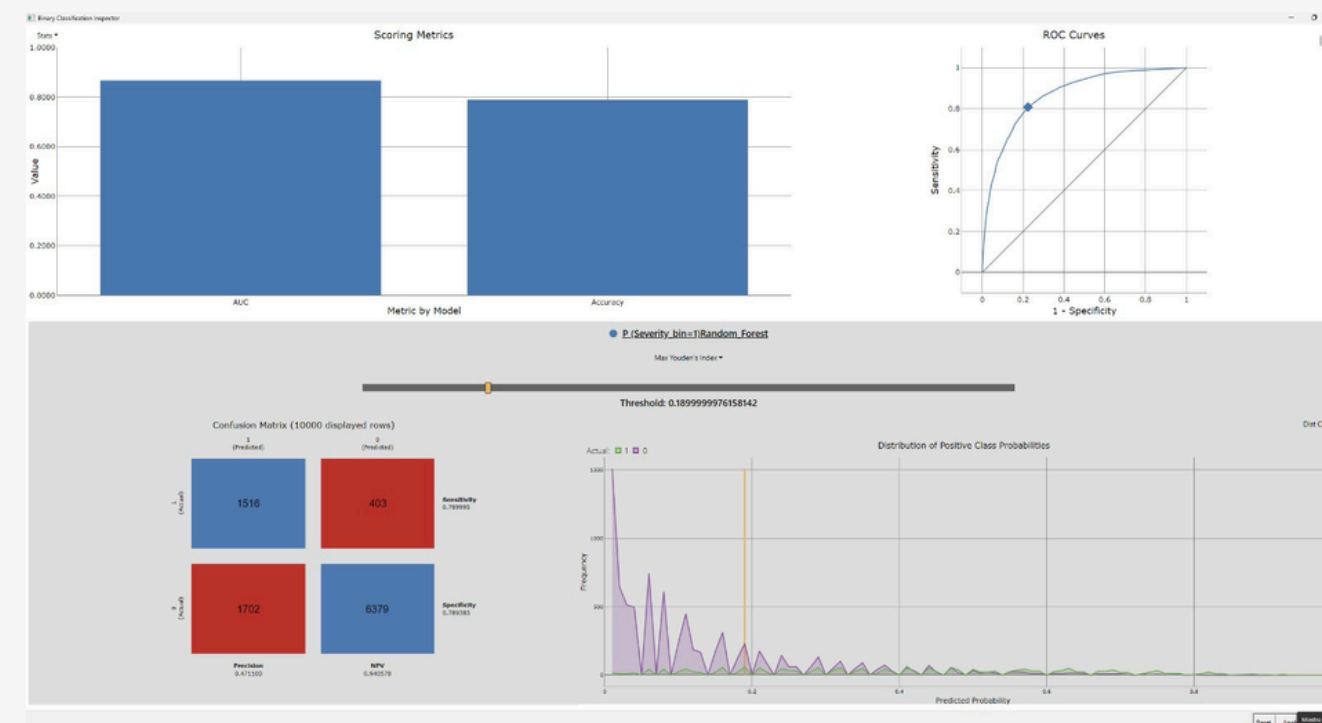
Math Formula in each of the three branches calculates a simple structural metric of the model - the total number of splits of the tree at levels 0, 1, and 2.

The formula:

$\$ \# \text{ splits}(\text{level } 0) \$ + \$ \# \text{ splits}(\text{level } 1) \$ + \$ \# \text{ splits}(\text{level } 2) \$$
sums up how many times the tree splits the data at the key first levels. This does not affect the forecast itself, but adds an indicator of model complexity: the more splits, the more intensively the model searches for patterns. Each branch adds its own column so that the complexity of the three versions of Random Forest can be compared with each other. All formulas are the same, only the column names differ for ease of analysis.



Basic Random Forest

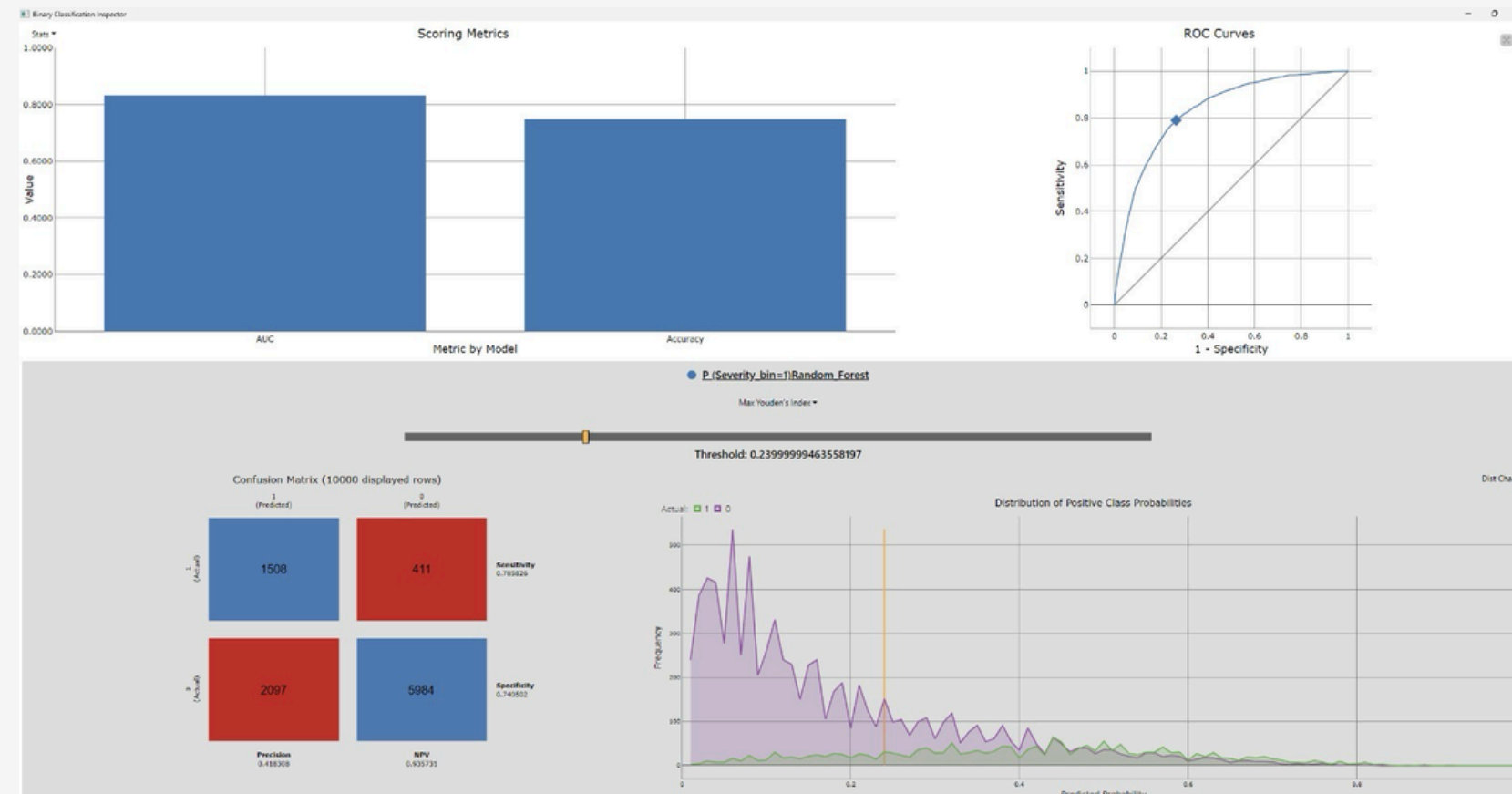


The unbalanced Random Forest achieves the best overall performance among the three specifications. With $AUC \approx 0.86$, the model demonstrates strong discriminative ability and separates severe vs. non-severe accidents more effectively than the balanced or reduced models. Sensitivity remains high ($\approx 78\%$), meaning the model successfully detects most severe accidents without oversampling. At the same time, precision ($\approx 47\%$) and specificity ($\approx 79\%$) are clearly higher than in the balanced version, indicating fewer false alarms and better recognition of non-severe events.

The confusion matrix confirms this: false positives decrease substantially while maintaining nearly the same recall as the SMOTE model. The selected threshold (≈ 0.19) provides a stable balance between sensitivity and specificity.

In summary, this is the most reliable and efficient model, combining strong predictive accuracy with lower operational cost due to reduced false positives.

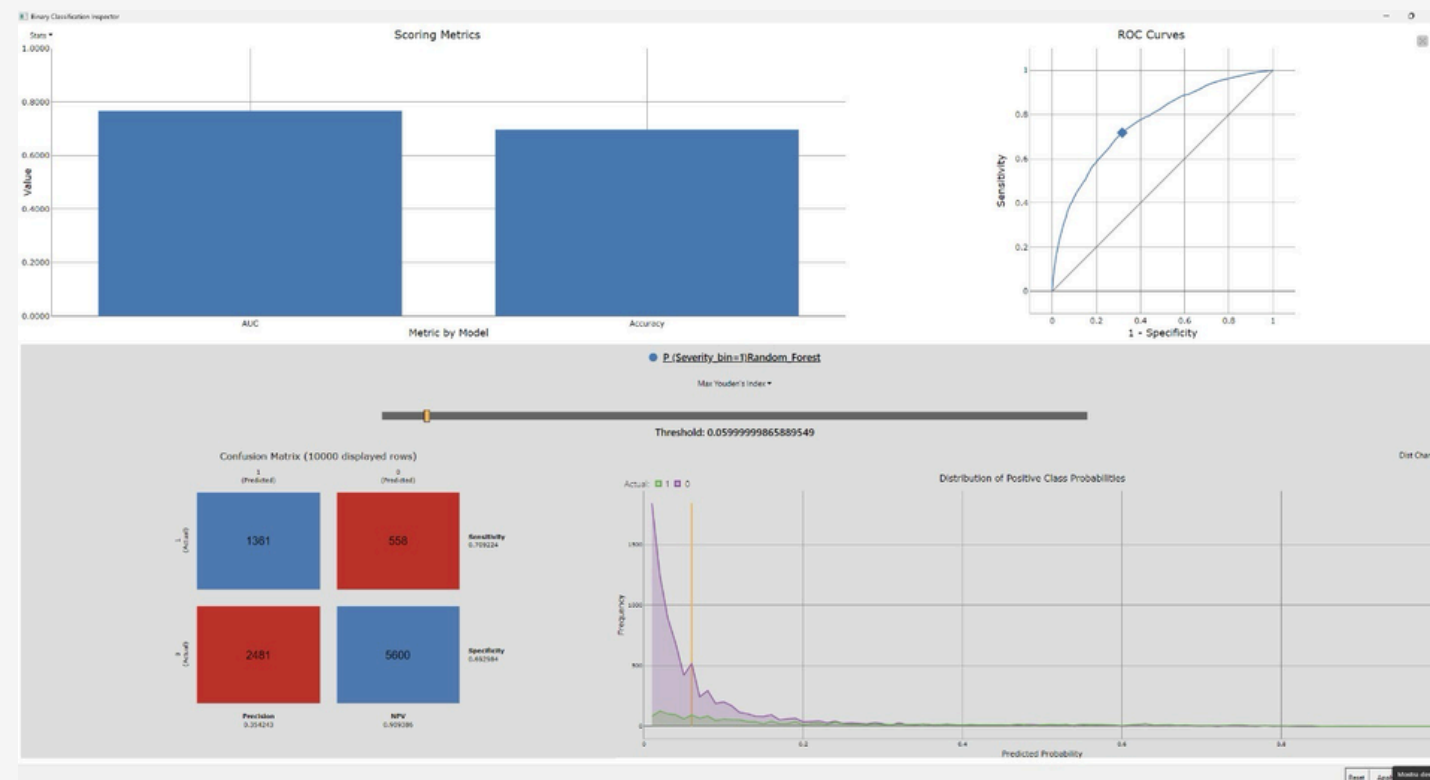
Balanced Random Forest



The balanced Random Forest, trained on SMOTE-augmented data, achieves strong sensitivity ($\approx 78.6\%$)—meaning it correctly captures most severe accidents. This confirms that oversampling effectively shifts the model toward prioritizing minority-class recognition. However, precision is low ($\approx 41\%$), indicating that many predicted severe accidents are actually non-severe, producing a high false-positive rate. Specificity remains moderate ($\approx 74\%$), reflecting this trade-off. The overall performance, reflected by $AUC \approx 0.83$, shows good class separation but lower discriminative power than the base model.

This configuration is safety-oriented: it reduces the risk of missing a severe accident, even at the cost of generating more false alarms. It is valuable when recall is strategically more important than operational cost.

Random Forest WITHOUT Correlated Variables



Removing correlated and harder-to-interpret variables substantially reduces the predictive performance of the Random Forest.

The model still captures around 70% of severe accidents (recall), but the drop in precision to 35% shows a very high rate of false alarms: more than 2 out of 3 predicted severe accidents are actually non-severe. Accuracy ($\approx 65\%$) and specificity ($\approx 65\%$) also decline markedly compared to the basic and balanced models. AUC falls to ≈ 0.75 , indicating noticeably weaker separation between the two classes.

The extremely low threshold (≈ 0.06) selected by Youden's index reflects the model's struggle: it must lower the decision boundary drastically to preserve sensitivity, which inflates false positives.

Overall, this version is more interpretable but far less reliable. It demonstrates that removing geographic and meteorological predictors causes a significant loss of signal, and even 500 trees cannot compensate for reduced feature richness.

Comparison of Random Forest Specifications

BASIC RANDOM FOREST - BASE MODEL

The Basic Model serves as our primary Random Forest specification. It achieves an AUC of 0.867 and an accuracy of 0.789, making it the strongest performer among the three. This version uses the full feature set produced after data preparation, without any additional balancing or variable reductions. Its superior results indicate that Random Forest benefits from having access to the complete set of geographic, temporal, and weather-related predictors. The model effectively captures severity patterns in the national dataset and provides a stable benchmark for evaluating alternative configurations.

BALANCED RANDOM FOREST

The Balanced Model, created by increasing the representation of severe accidents using SMOTE and row sampling, achieves an AUC of 0.833 and accuracy of 0.749. Although weaker than the Basic Model, it performs substantially better than the Reduced-Variables version. The improvement in sensitivity (visible in the confusion matrix) reflects the benefits of rebalancing in identifying rare severe cases. However, this gain comes at the cost of slightly lower overall accuracy and ranking ability, as oversampling increases noise in the training distribution.

REDUCED VARIABLES MODEL

The variant that removes correlated variables performs noticeably worse, with an AUC of 0.766 and accuracy of 0.696. Eliminating key weather and geographic indicators reduces the model's ability to differentiate between severe and non-severe accidents. This confirms that even correlated variables still contribute meaningful information for tree-based methods, which are robust to multicollinearity. The decline in metrics suggests that important contextual signals—especially those related to location and environmental conditions—were lost in the simplified feature set.

OVERALL ASSESSMENT

The comparison shows that the Basic Random Forest remains the preferred specification, offering the best balance between accuracy, class-ranking ability (AUC), and model stability. The Balanced Model is useful when prioritizing detection of severe accidents, but the added noise reduces its overall predictive power. In contrast, the Reduced-Variables Model demonstrates that removing correlated predictors weakens the model substantially, reinforcing the importance of retaining the full feature set for this task. These findings highlight Random Forest's robustness but also show that its performance is sensitive to feature richness and class distribution. This explains why the Basic Model serves as the foundation for comparisons with Gradient Boosting and XGBoost in the rest of the analysis.

Comparison of Variable Importance Across Specifications

To understand how the Random Forest model responds to different data and feature configurations, we compare the variable importance rankings of the three specifications: Basic, Balanced, and Reduced-Variables. Although all three share the same underlying algorithm, their importance patterns shift depending on feature availability and sampling strategy. These differences help identify which predictors are structurally important and which change when the model setup is altered. The Basic Random Forest, which is our main specification, consistently identifies the same core set of predictors as most influential. State is by far the dominant variable, followed by Distance(mi), Wind_Direction, Traffic_Signal, Precipitation_missing, Crossing, Station, and the geographic coordinates Start_Lat and Start_Lng. Time-related features such as Start_Time_hour and Start_Time_dayOfWeek also appear, but with lower impact. Because this model uses the full feature set, it captures the geographic, infrastructural, and weather-related patterns that drive accident severity across the United States. This specification produces the most stable and interpretable results and therefore serves as the preferred model.

In the Balanced Random Forest, where severe accidents are upsampled, variable importance becomes more evenly distributed across predictors. State remains the top feature, but with less dominance than in the Basic Model. Distance(mi) and Traffic_Signal increase in relative importance, while some weather variables and Wind_Direction decline slightly. Oversampling reduces the model's reliance on broad geographic grouping and pushes it to learn more fine-grained distinctions in the data. However, this redistribution of importance also corresponds to the reduced overall predictive performance of the Balanced Model.

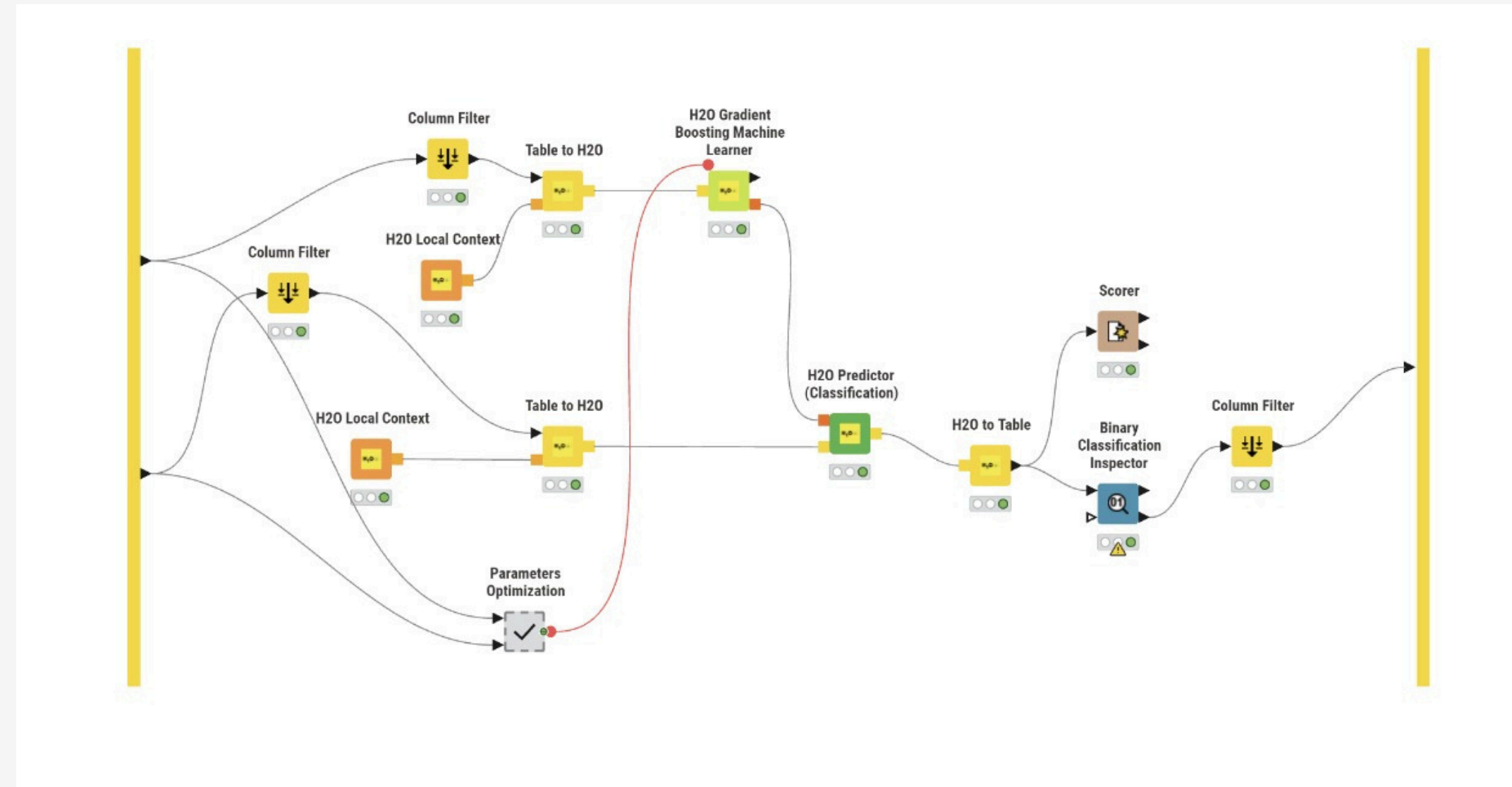
Comparison of Variable Importance Across Specifications

The Reduced-Variables Random Forest, which removes correlated predictors, shows a clear decline in model quality and a narrowing of variable importance. State remains the most influential variable, followed by Distance(mi), but many weather and geographic variables drop sharply because their correlated counterparts have been removed. Variables such as Traffic_Signal, Crossing, Wind_Speed, and Station appear artificially more important simply because competing features are no longer available. This confirms that removing correlated predictors strips away useful information, since Random Forest models are naturally robust to multicollinearity.

Overall, across all three specifications, State and Distance(mi) consistently emerge as the most important predictors, highlighting the strong geographic structure underlying accident severity. The Basic Model produces the most balanced and meaningful importance distribution, while the Balanced and Reduced models reveal how sensitive Random Forest can be to data composition and feature richness. These results reinforce the choice of the Basic Random Forest as the main specification for interpretation and comparison in this project.

4.2 - H2O Gradient Boosting

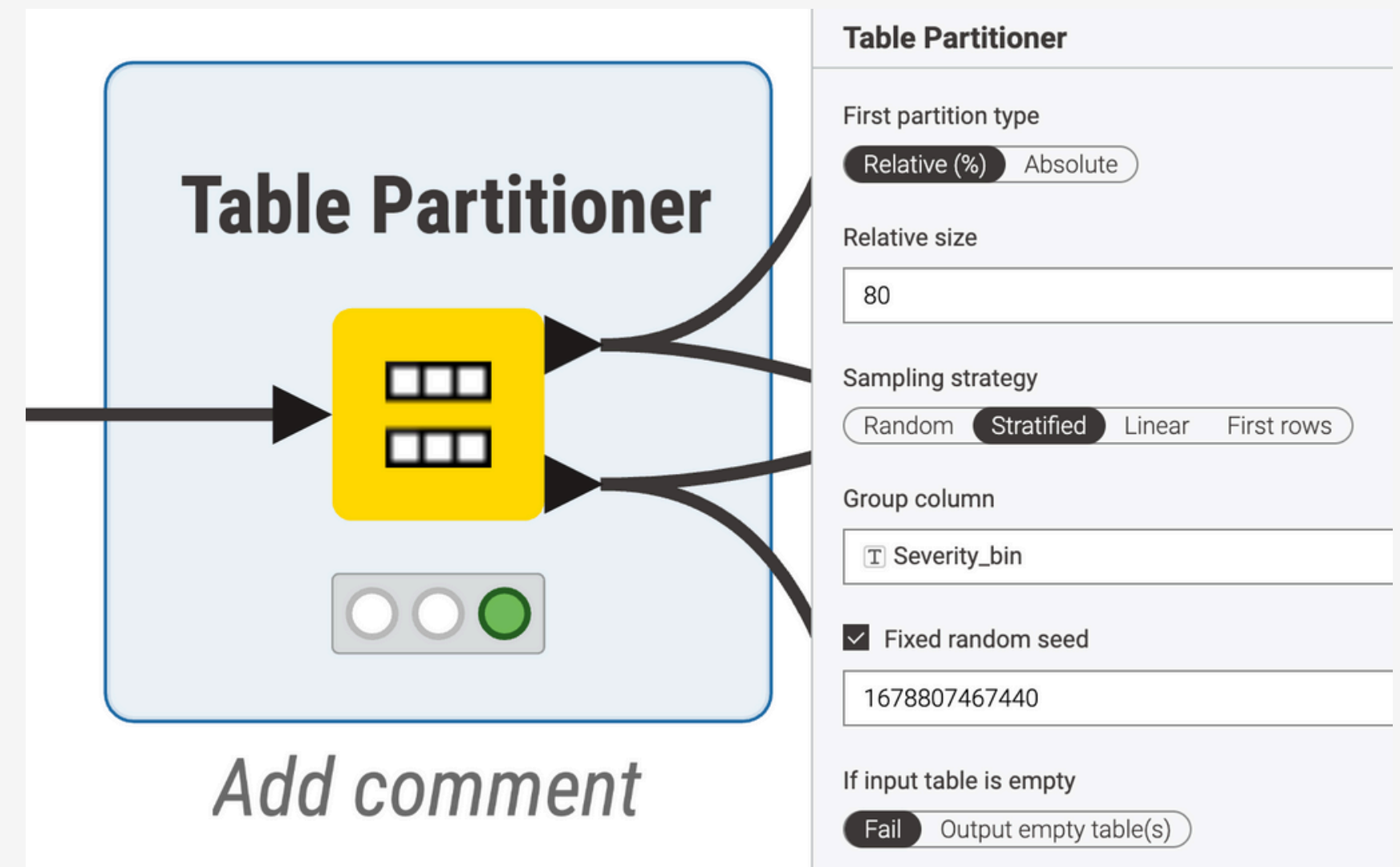
- Data input & Partitioning
- Feature selection & H2O conversion
- Hyperparameter optimization loop
- Model training
- Prediction & Model evaluation
- Variable importance (interpretability)



Data input & Partitioning

TABLE PARTITIONER

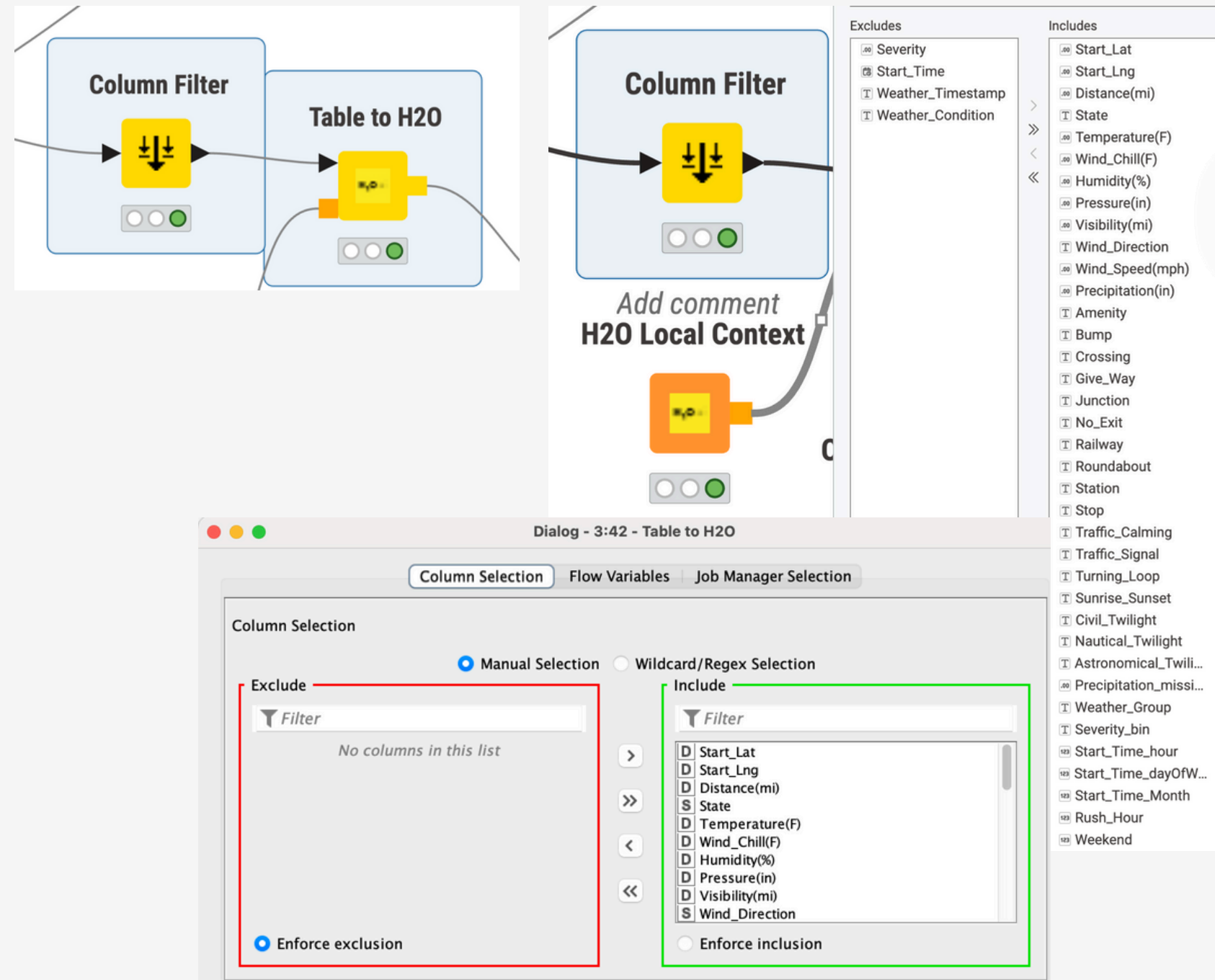
After completing data preparation, the data set was divided into training and test or sample sets using Table Partitioner. Stratified sampling was used to maintain the proportion of Severe/Non-Severe classes in both parts of the sample. The training sample accounts for 80% of the data, and the test sample accounts for 20%. Setting a fixed ratio ensures the reproducibility of results. This approach guarantees correct model training and independent verification of its performance on deferred data.



Feature selection & H2O conversion

COLUMN FILTER AND TABLE TO H2O

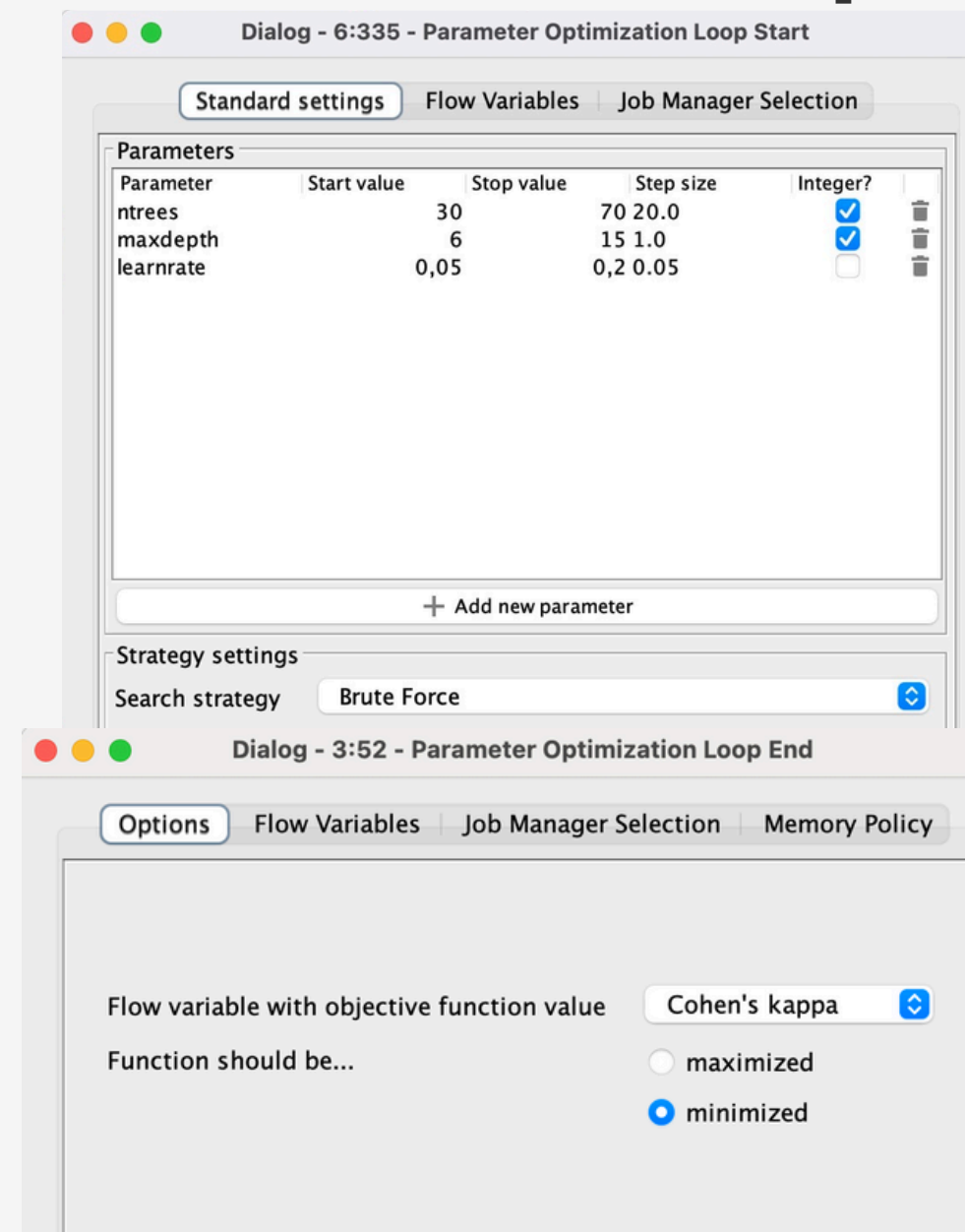
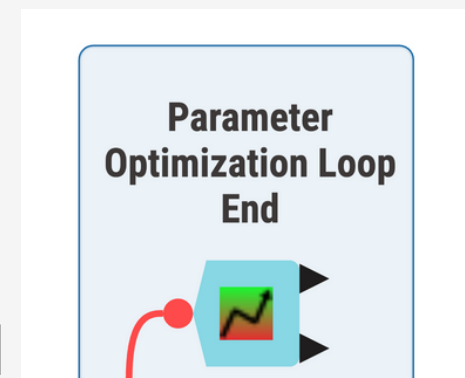
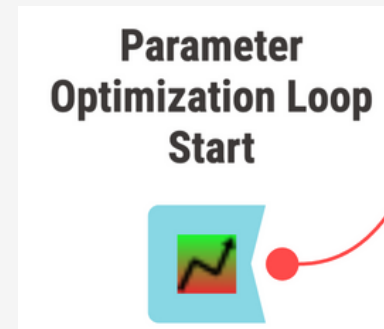
Before transferring data to the H2O model, all relevant features are selected using Column Filter. The selection includes geographic, weather, infrastructure, and numerical variables, as well as indicators created during data preparation (Weather_Group, Rush_Hour, Weekend, etc.). Table to H2O node converts the table to H2OFrame format while preserving the structure and type of variables. This allows the use of a full set of engineering features when training the Gradient Boosting model.



Hyperparameter optimization loop

HYPERPARAMETER OPTIMIZATION LOOP START
HYPERPARAMETER OPTIMIZATION LOOP END

To improve model performance, a Hyperparameter Optimization Loop was used, which iterates through combinations of Gradient Boosting parameters within specified ranges. The cycle changed ntrees (30-70), maxdepth (6-15), and learnrate (0.05-0.2), and the quality of each model was evaluated using the minimized Cohen's kappa indicator. The results of the search showed that the optimal parameters were: ntrees = 70, maxdepth = 15, learnrate = 0.15, which provided the best objective function value (0.858). These parameters were used to build the final model.



Dialog - 3:52 - Parameter Optimization Loop End

Flow variable with objective function value: Cohen's kappa

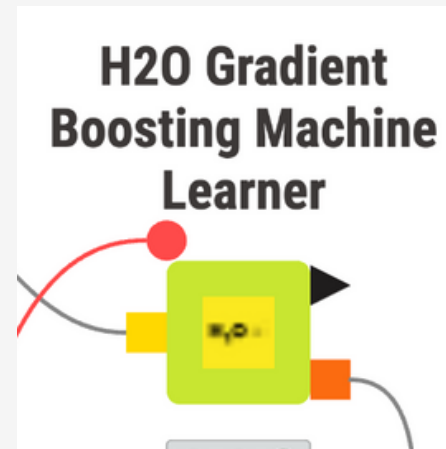
Function should be...: ☒ minimized

#	RowID	ntrees Number (Integer)	maxdepth Number (Integer)	learnrate Number (Float)	Objective value Number (Float)
1	Best p	70	15	0.15	0.858

Model training

H2O GRADIENT BOOSTING MACHINE LEARNER

The H2O Gradient Boosting Machine Learner model builds an ensemble of weak decision trees, where each subsequent tree corrects the errors of the previous ones. Severity_bin is used as the target variable. The model includes all selected weather, time, geographic, and infrastructure features. Key hyperparameters, such as tree depth, learning rate, and number of models, determine the complexity and ability of the model to generalize patterns. GBM is robust to nonlinearities and interactions between features, making it effective for predicting the severity of traffic accidents.

A screenshot of the H2O Gradient Boosting Machine Learner configuration dialog. The dialog has four tabs: General Settings, Algorithm Settings, Advanced Settings, and Flow Variables. The Target Column is set to Severity_bin. The Exclude list is empty, and the Include list contains Start_Lat, Start_Lng, Distance(mi), State, Temperature(F), Wind_Chill(F), Humidity(%), Pressure(in), and Visibility(mi). The Tree Options section shows Number of levels (tree depth) set to 10. The Boosting Options section shows Number of models set to 200 and Learning rate set to 0.1. The Advanced Settings section shows Min (weighted) observations set to 10, Min relative split improvement rate set to 0.00001, Row sample rate (per tree) set to 1, Column sample rate (per tree) set to 1, Relative change of column sample rate per level set to 1, Histogram type set to Auto, Number of histogram bins (numerical) set to 20, Number of histogram bins (categorical) set to 1 024, Max number of root histogram bins (numerical) set to 1 024, Column sample rate (global) set to 1, Learning rate annealing set to 1, Distribution set to Auto, Max absolute value of a leaf node prediction set to 0, and Bandwidth of Gaussian multiplicative noise set to 0.

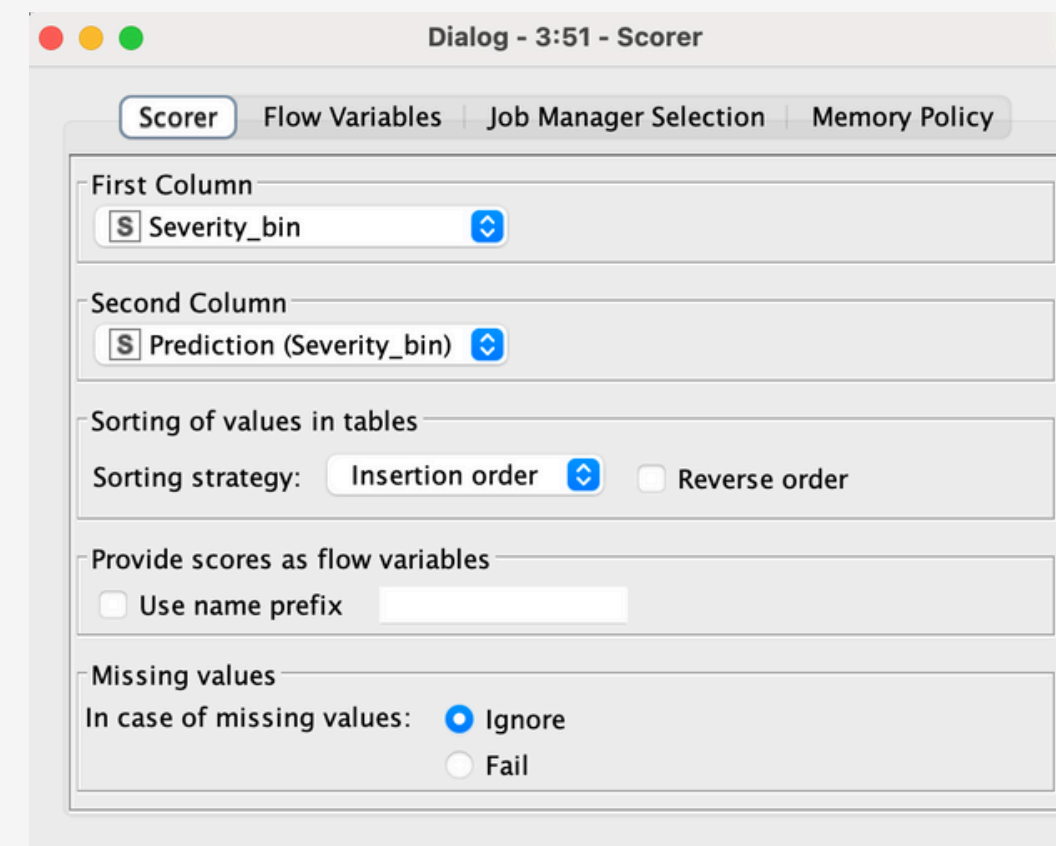
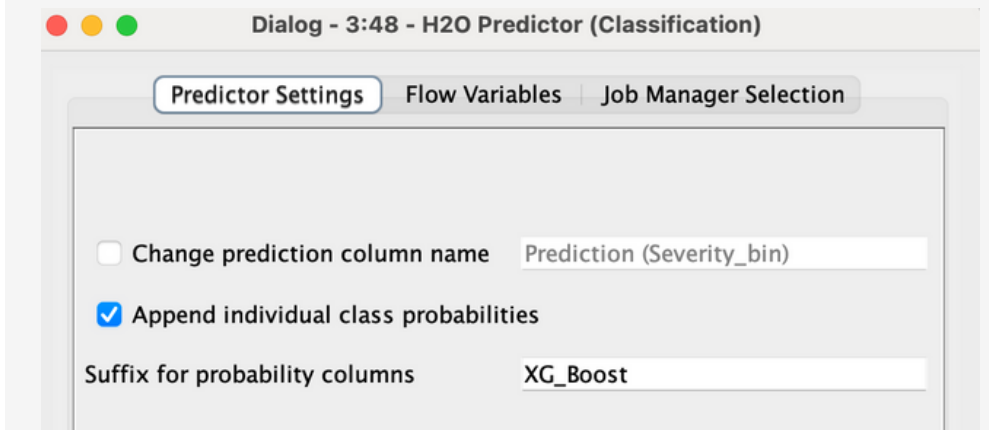
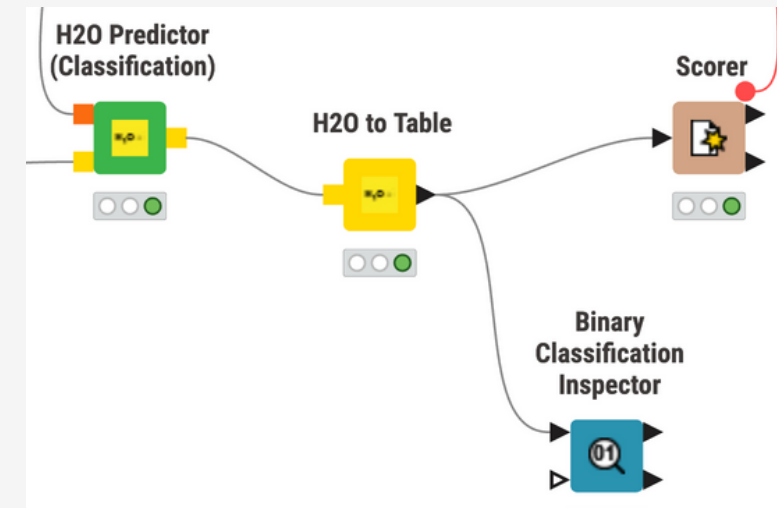
Prediction & Model evaluation

H2O PREDICTOR

SCORER

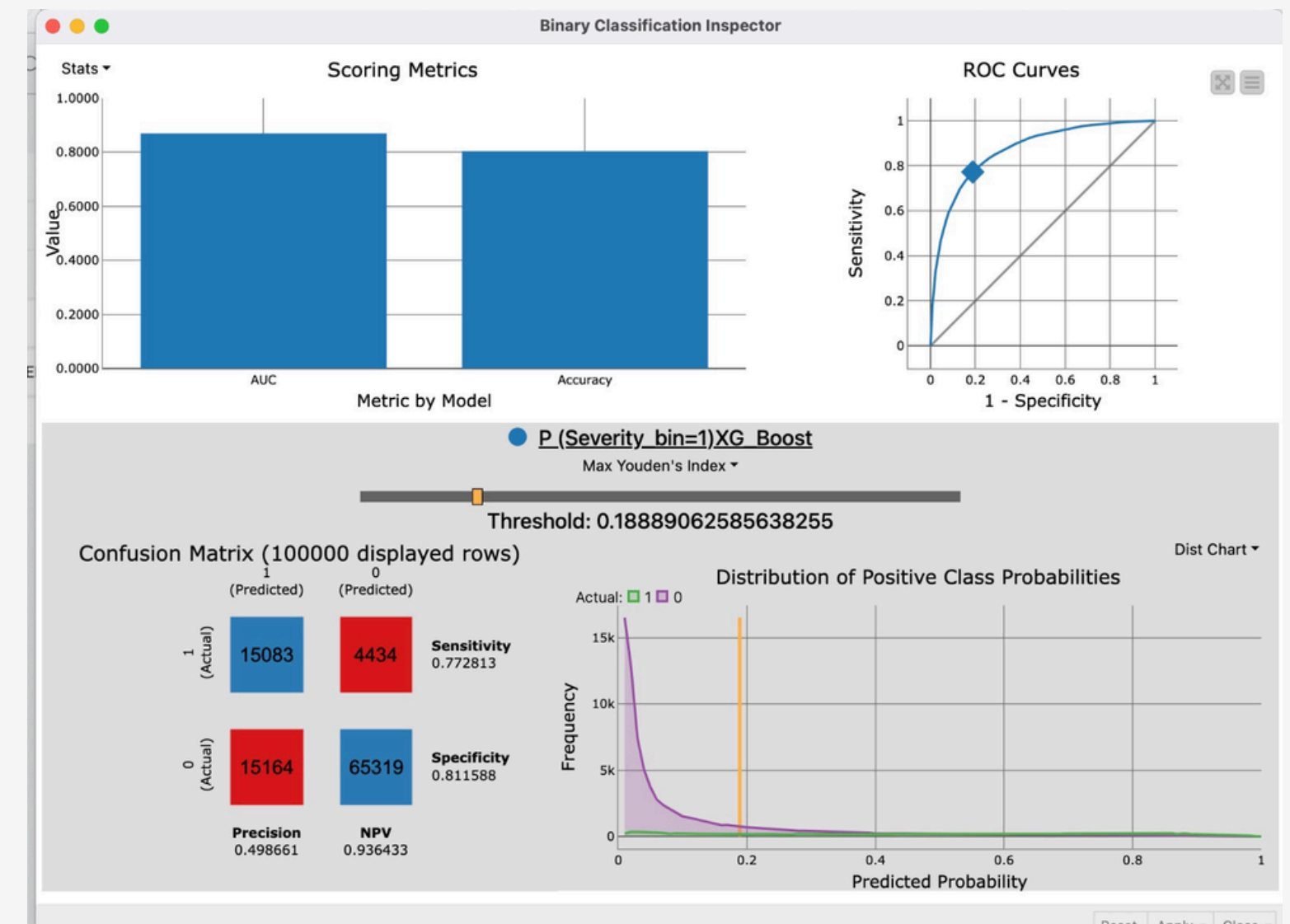
BINARY CLASSIFICATION INSPECTOR

The Scorer node performs a basic assessment of classification quality by comparing the actual Severity_bin values with those predicted by the model. Based on this, key performance metrics such as accuracy, precision, recall, and F-measure are calculated. The Scorer provides a basic qualitative summary before applying more detailed diagnostics in the next node, Binary Classification Inspector.



Binary classification inspector

The final Gradient Boosting model was evaluated using Binary Classification Inspector. The model demonstrates a high AUC (~ 0.87), indicating a good ability to distinguish between traffic accident severity classes. Accuracy and Balanced Accuracy (~ 0.78) indicate balanced prediction quality with uneven class distribution. Sensitivity (0.77) and specificity (0.81) form an effective balance between detecting serious cases and reducing false alarms. The optimal classification threshold is set at 0.1889 according to Youden's Index. The Confusion Matrix for 100,000 rows confirms the stable result of the model, and the probability distribution demonstrates good class separation.



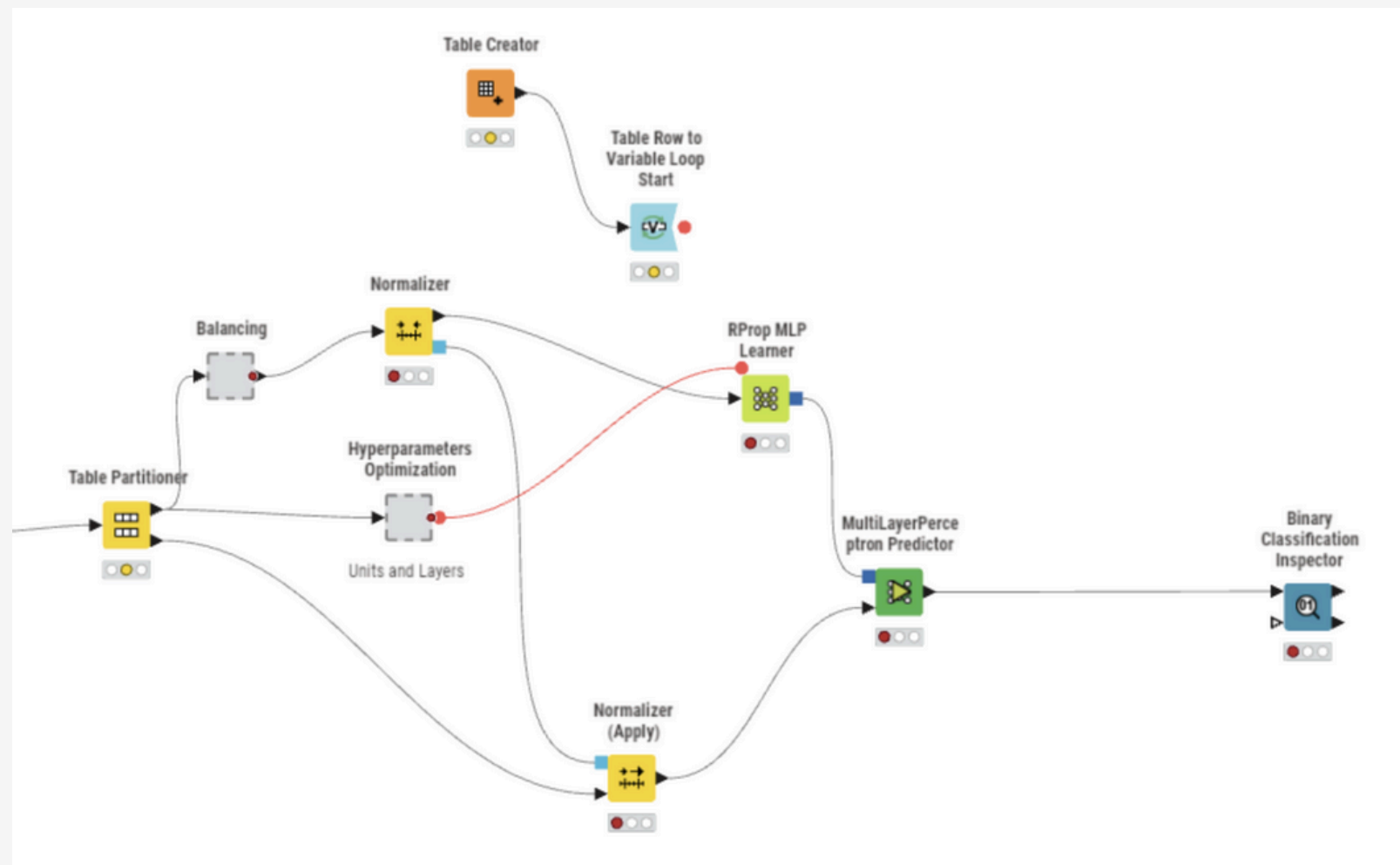
Variable importance (interpretability)

To interpret the model, we used a feature importance table generated by H2O GBM. In it, each variable is assigned a relative importance that reflects its contribution to improving the accuracy of the forecast.

The most influential factors were geographic coordinates, travel distance, precipitation intensity, and certain time and infrastructure characteristics. The most influential factors were geographic coordinates, travel distance, precipitation rate and intensity, as well as certain time and infrastructure characteristics. Analysis of feature importance allows us to understand which conditions are most associated with an increased risk of serious accidents.

Rows: 36 Columns: 3					
☐	#	RowID	Relative Importance Number (Float)	Scaled Importance Number (Float)	Percentage Number (Float)
☐	1	State	27,732.75	1	0.192
☐	2	Distan	21,930.242	0.791	0.152
☐	3	Wind_I	15,193.227	0.548	0.105
☐	4	Precipi	14,206.268	0.512	0.098
☐	5	Start_L	11,509.596	0.415	0.08
☐	6	Traffic	10,928.311	0.394	0.076
☐	7	Start_L	10,759.122	0.388	0.074
☐	8	Start_T	5,520.848	0.199	0.038
☐	9	Start_T	3,826.82	0.138	0.026
☐	10	Crossir	3,324.063	0.12	0.023
☐	11	Pressu	2,765.609	0.1	0.019
☐	12	Start_T	2,495.547	0.09	0.017
☐	13	Wind_C	1,928.595	0.07	0.013
☐	14	Stop	1,863.202	0.067	0.013
☐	15	Humid	1,768.675	0.064	0.012
☐	16	Juncti	1,668.706	0.06	0.012
☐	17	Weathr	1,514.703	0.055	0.01
☐	18	Tempe	1,429.253	0.052	0.01
☐	19	Wind_S	953.702	0.034	0.007
☐	20	Visibili	533.105	0.019	0.004
☐	21	Station	470.099	0.017	0.003
☐	22	Astron	427.653	0.015	0.003
☐	23	Rush_T	375.169	0.014	0.003
☐	24	Civil_T	223.721	0.008	0.002
☐	25	Nautic	216.922	0.008	0.002
☐	26	Precipi	197.85	0.007	0.001
☐	27	Sunrise	182.241	0.007	0.001
☐	28	Amenit	172.653	0.006	0.001
☐	29	Railwa	161.697	0.006	0.001
☐	30	Give_W	111.856	0.004	0.001
☐	31	Weeke	77.423	0.003	0.001
☐	32	Traffic	26.319	0.001	0
☐	33	No_Ext	6.102	0	0
☐	34	Bump	0.739	0	0
☐	35	Round	0	0	0
☐	36	Turnin	0	0	0

4.3 Multi-Layer Perceptron (MLP) Algorithm

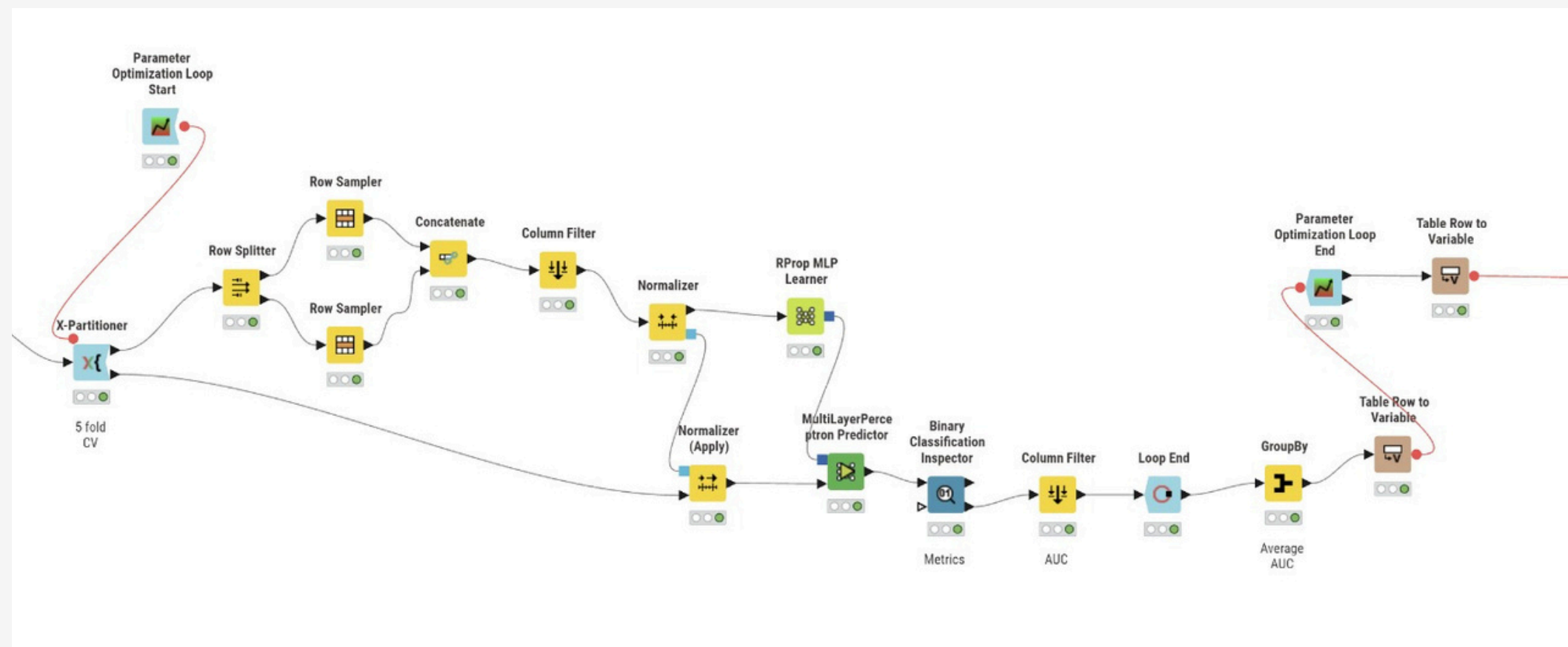


The MLP was first tested on the full U.S. dataset, but its performance was very poor due to the high geographic and infrastructural heterogeneity across states. Neural networks are sensitive to such variability, and a single national model could not learn consistent patterns. Since the tree-based models had already shown State to be one of the strongest predictors of severity, we shifted to a state-specific MLP approach, training a separate model for each state. This provides more homogeneous data for learning, improves model performance, and allows us to examine how accident severity behaves across different regions.

The workflow, preprocessing steps, and hyperparameter optimization described below are therefore applied on a state-by-state basis, with detailed results shown for Texas and New York.

Workflow Structure

The MLP workflow follows the same general modeling structure used for the other algorithms—train–test split, balancing, normalization, training, and evaluation—but it is implemented on a state-by-state basis. After selecting a state with a Row Filter, the data is partitioned into training and test sets, preserving the local class imbalance. The training portion is then balanced using a down-sampling procedure and normalized to ensure stable neural network training.

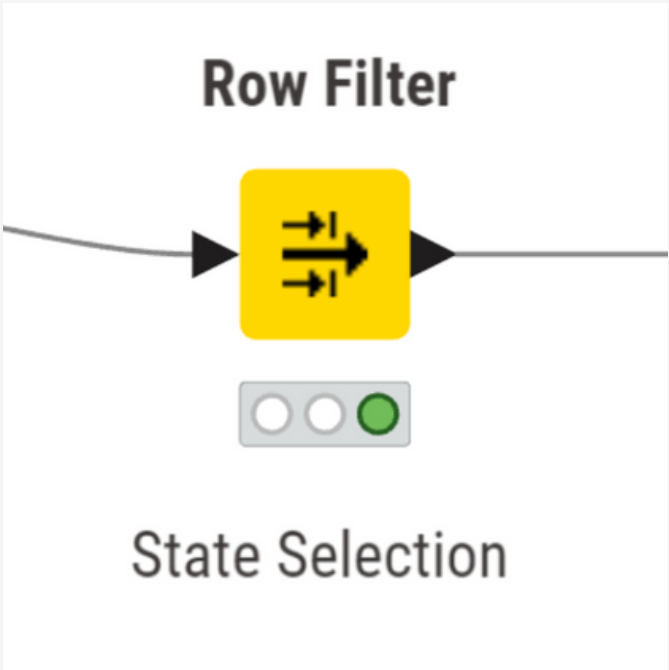


A dedicated hyperparameter optimization metanode evaluates multiple network architectures by combining different numbers of hidden layers and neurons. Each configuration is assessed through 5-fold cross-validation, allowing us to measure performance across several training/validation splits. The architecture with the highest average AUC is passed to the main workflow, where the final MLP model is trained on the complete balanced dataset for that state.

Data Preparation for MLP

STATE SELECTION

To the purpose of state-based analysis at the beginning of this model is possible to select a single state by inserting the state code in the Row Filter node configuration.



Criterion 1

Filter column

Operator

State

Equals

Case matching

Case sensitive

Case insensitive

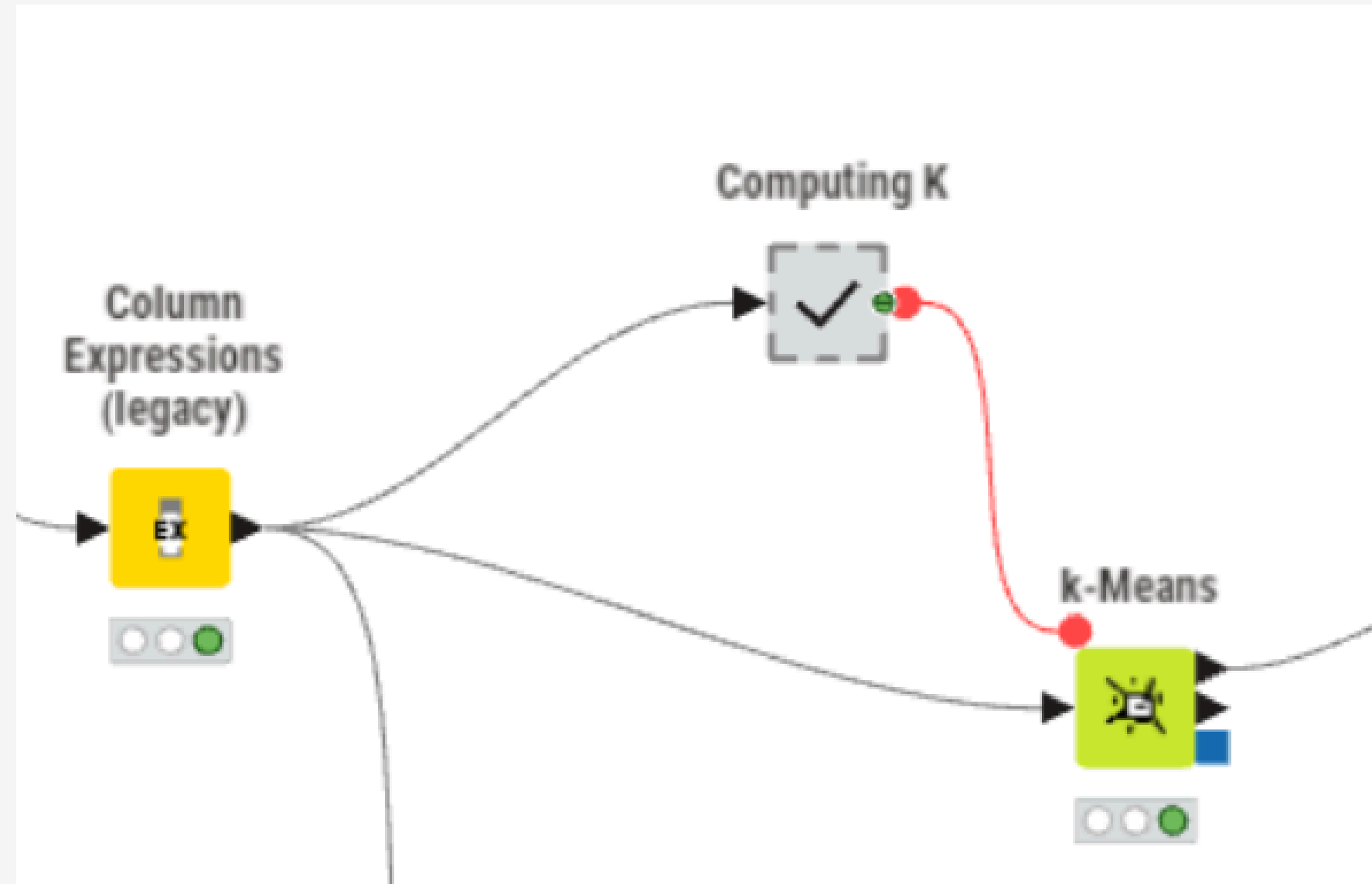
Value

NY

⊕

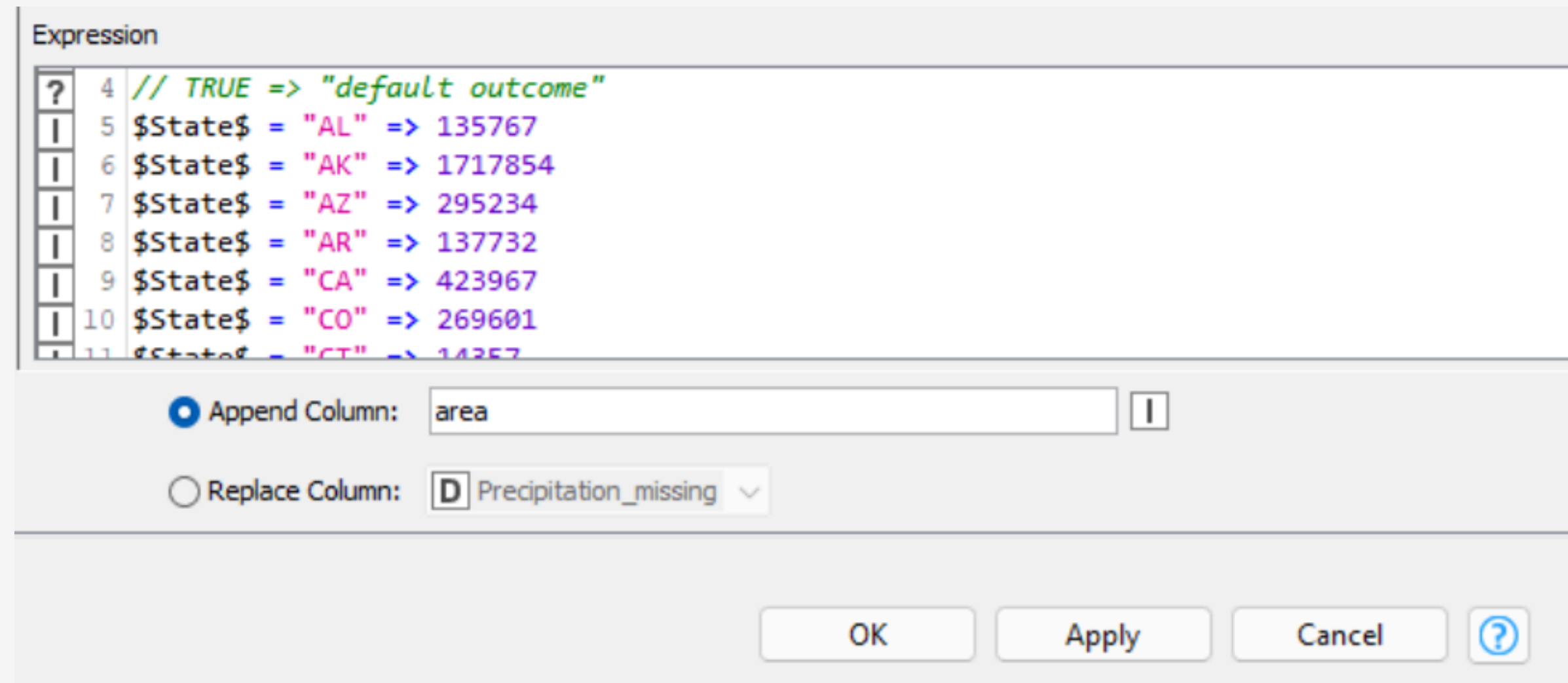
Add filter criterion

Data Preparation for MLP - K-Means



K-means is a partitioning clustering algorithm that groups observations into K clusters by iteratively assigning each point to the nearest centroid and then updating the centroids as the mean of their assigned points. The choice of K is not fixed a priori and strongly influences the quality and granularity of the resulting clusters. In our setting, we cannot use a single global value of K , because the spatial extent and data density vary substantially across states. Instead, we assign a different number of clusters per state, taking into account both the state's geographic size and the number of observations within it, so that the clustering remains informative but computationally manageable.

Data Preparation for MLP: K-Means



To pursue this goal we started by adding a feature in the dataset through the Rule Engine node. The new feature is named "area" and the true area of the state of reference is inserted in the unit of squared kilometers.

Rule Engine



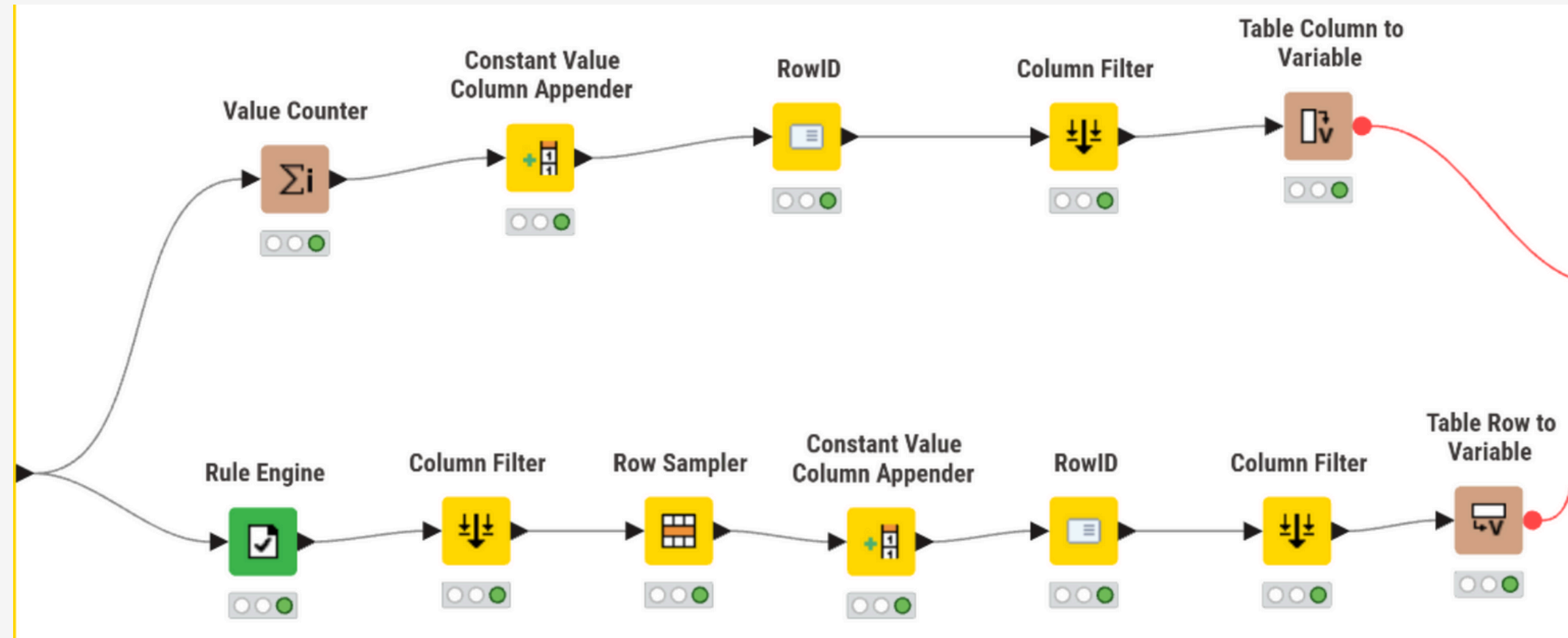
Data Preparation for MLP: K-Means

EXTRACTING NUMBER OF OBSERVATION

The extraction of the number of observations relates to the upper part of this code. Value Counter returns the number of rows in the dataset which are made readable as variable, for the computation of the number of clusters K in the equation, in following three nodes.

EXTRACTING AREA SIZE

The lower part of this code relates to the extraction of the area size to the same purpose. Column Filter extracts only the “area” feature, Row sampler reduces the rows to 1, the next three nodes have the same scope as before.



Data Preparation for MLP: K-Means

EQUATION FOR THE COMPUTATION OF THE NUMBER OF CLUSTERS

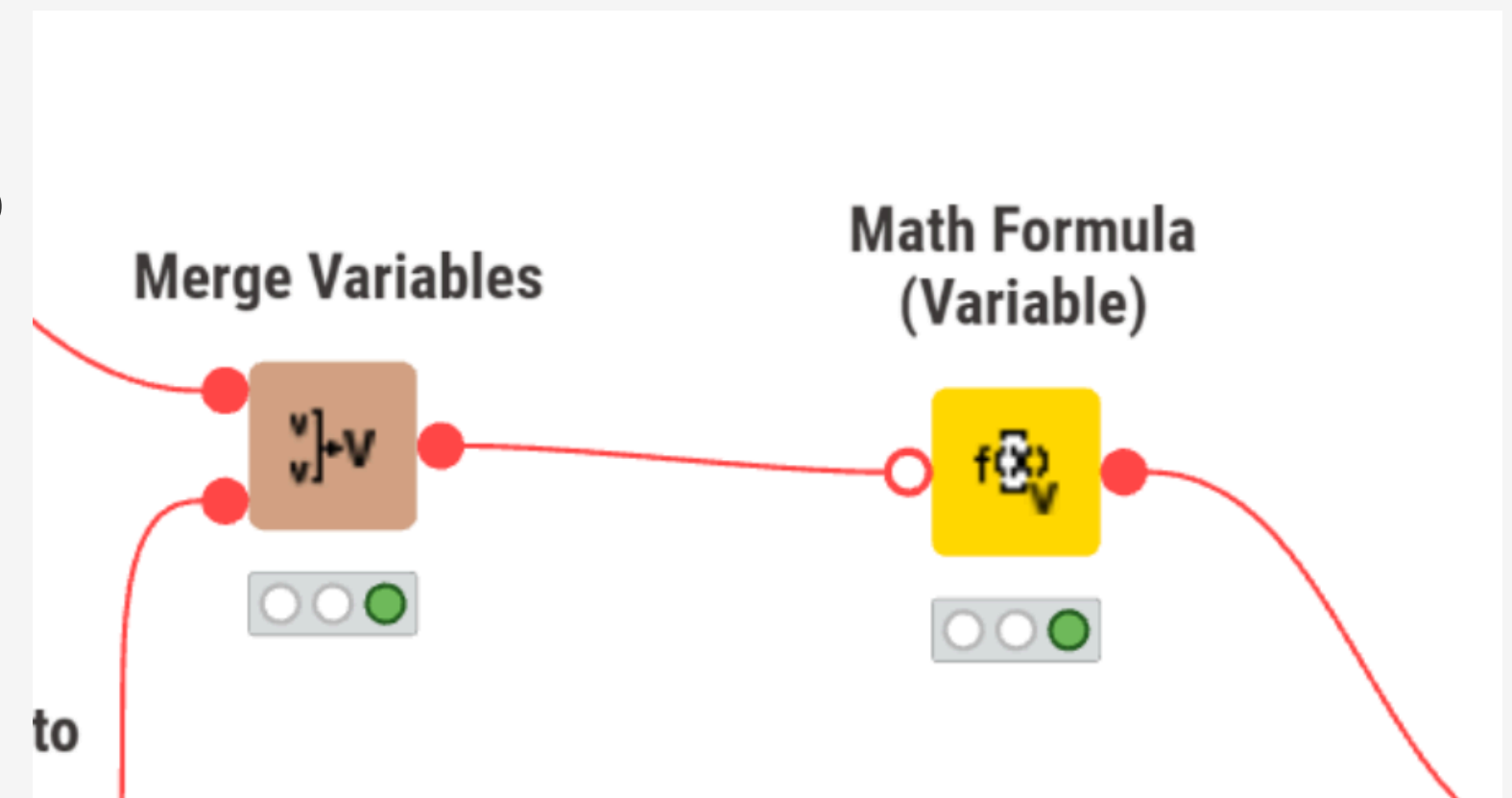
$$K = \text{clip}\left(\frac{N \cdot A}{100000}, 160, 10000\right)$$

- N is the number of observations per state
- A is the total area of the state in squared kilometres
- K is the number of cluster needed which is passed as variable to the K-means node

An upper bound of 10000 is placed to prevent for exagerous and not needed computational costs.

On the other hand less than 160 clusters is needed to control large areas such the ones of the states.

The resulting cluster ID is added as a feature to capture spatial variation within the state and improve the MLP's ability to learn local patterns.

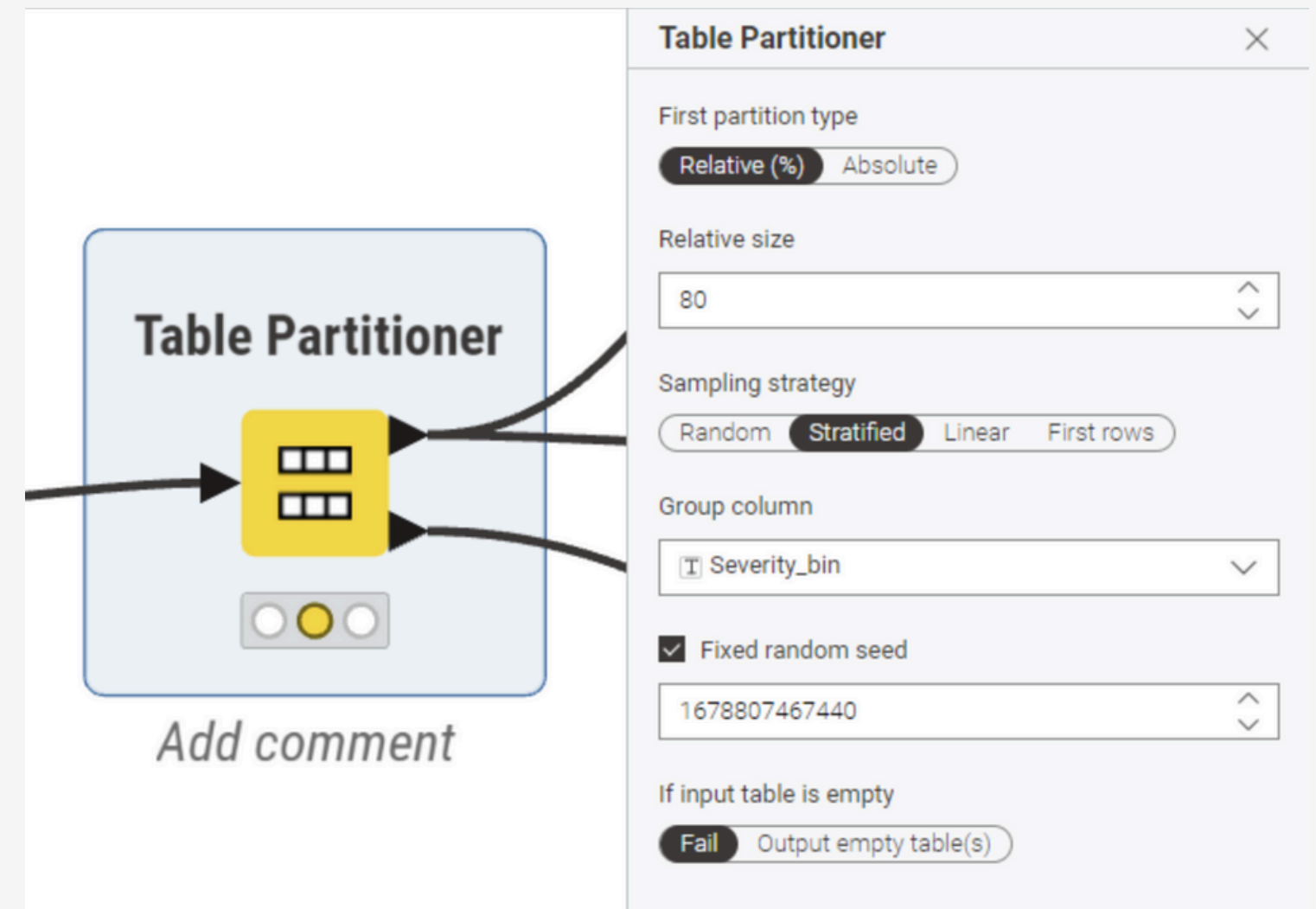


Data Preparation for MLP

TABLE PARTITIONER

To ensure proper model evaluation, the dataset of ~500,000 observations and 46 variables is split into 80% training and 20% testing using stratified sampling on Severity_bin. This preserves the original class imbalance in both subsets, where severe accidents represent only a small fraction of the total cases.

The split guarantees that all preprocessing, balancing, and model training occur strictly on the training portion, keeping the test set fully unseen until final evaluation.

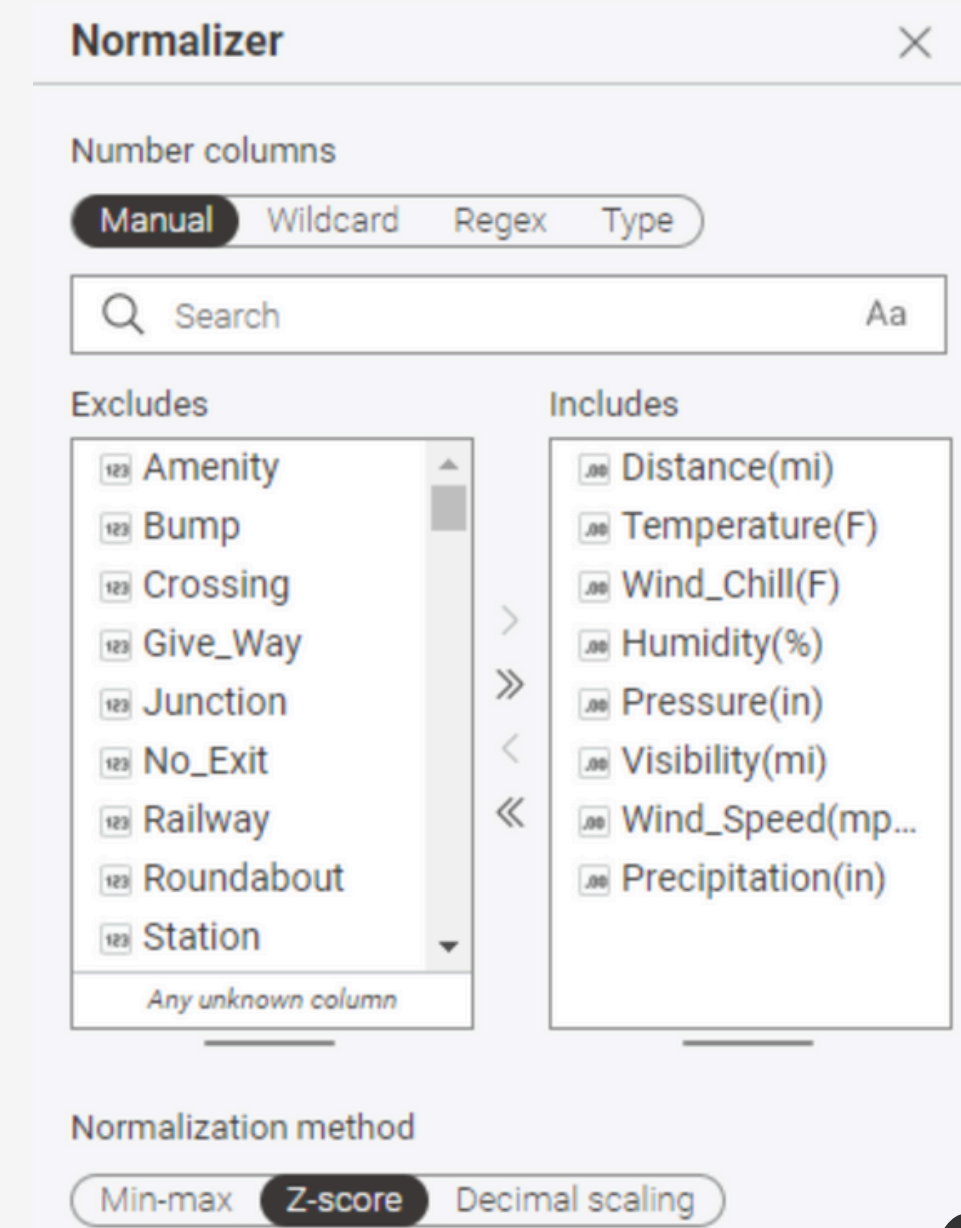


Data Preparation for MLP

NORMALIZER

Since neural networks are sensitive to the scale of the input variables, all numerical features are normalized before training the MLP. The Normalizer node computes the necessary statistics (mean and standard deviation) using only the balanced training set, and these parameters are then applied unchanged to the test set.

This prevents information from the test data from influencing the preprocessing step and ensures that both datasets follow the same scaling. Normalization stabilizes the training process and allows the MLP to update its weights more efficiently.



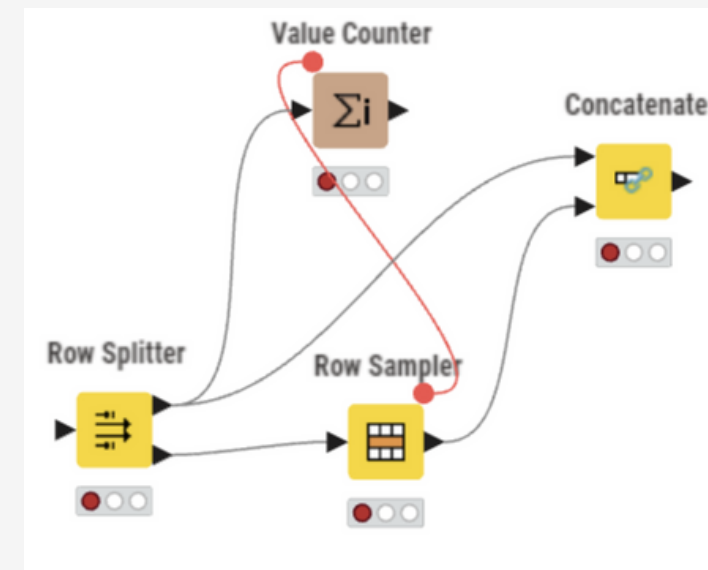
The screenshot shows the 'Normalizer' configuration window. At the top, there's a title bar with a close button. Below it, the 'Number columns' section has four tabs: 'Manual' (selected), 'Wildcard', 'Regex', and 'Type'. A search bar with a magnifying glass icon and the text 'Search' is present, followed by a small 'Aa' icon. The main area is divided into two columns: 'Excludes' and 'Includes'. The 'Excludes' list contains: Amenity, Bump, Crossing, Give_Way, Junction, No_Exit, Railway, Roundabout, and Station. The 'Includes' list contains: Distance(mi), Temperature(F), Wind_Chill(F), Humidity(%), Pressure(in), Visibility(mi), Wind_Speed(mp...), and Precipitation(in). Between the two lists are navigation arrows: a single right arrow (>), a double right arrow (>>), a single left arrow (<), and a double left arrow (<<). At the bottom, the 'Normalization method' section has three tabs: 'Min-max', 'Z-score' (selected), and 'Decimal scaling'.

Data Preparation for MLP

BALANCING METANODE

The target variable is imbalanced, with severe accidents accounting for only a minority of the dataset (estimates around 5–10%, depending on filtering). To prevent the model from ignoring the severe class, the training portion is balanced using a row-sampling procedure: the number of severe cases is counted and the majority class is randomly down-sampled to match it.

This produces a balanced dataset without creating synthetic observations and ensures that the model can learn patterns associated with severe accidents more effectively.



Row Splitter

Filter

Criterion 1

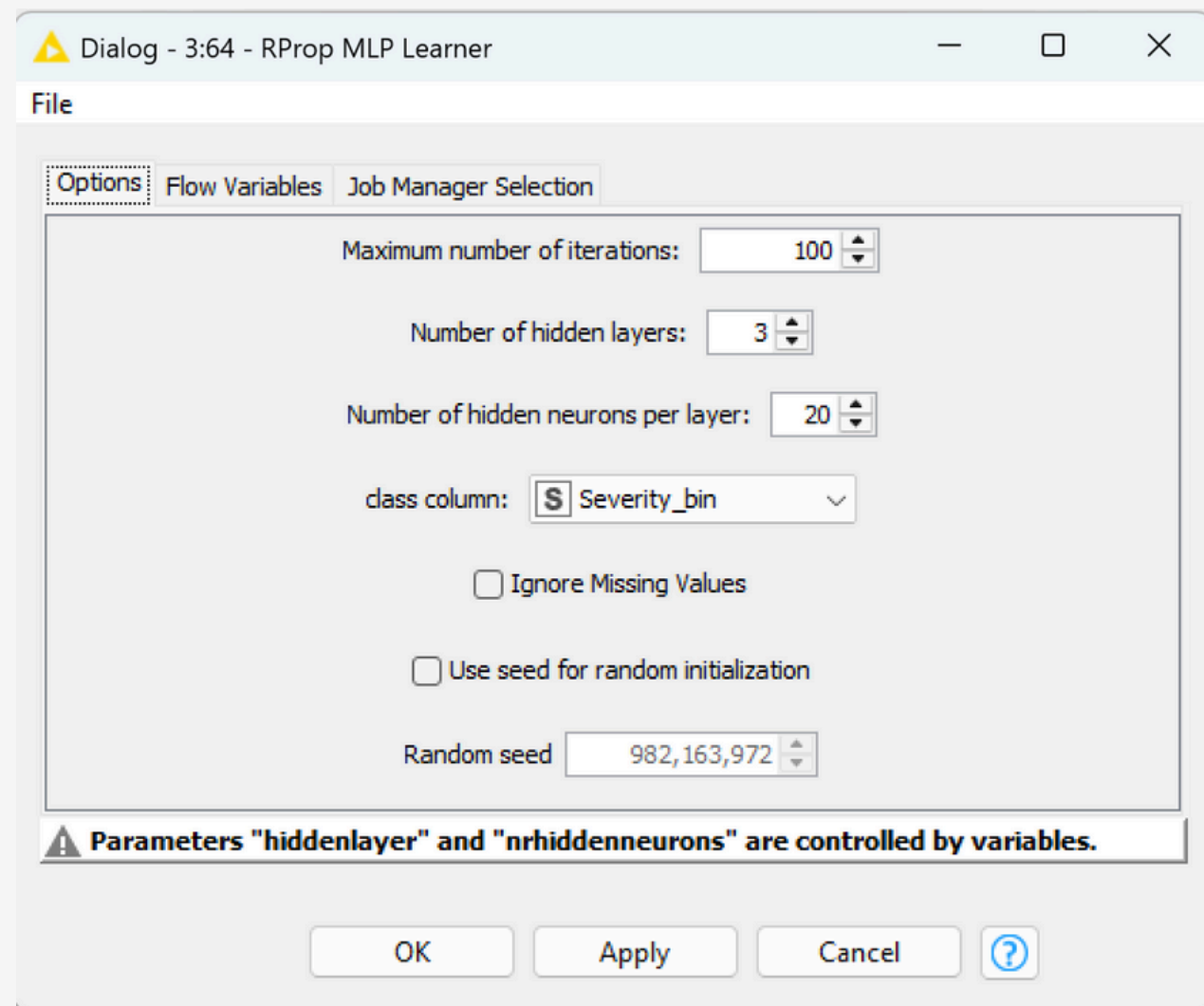
Filter columnSeverity_binOperatorEquals

Case matchingCase sensitiveCase insensitive

Value1

Model Training Pipeline

RPROP MLP LEARNER



The screenshot shows a software dialog box titled "Dialog - 3:64 - RProp MLP Learner". It has three tabs: "Options" (selected), "Flow Variables", and "Job Manager Selection". The "Options" tab contains the following settings:

- Maximum number of iterations: 100
- Number of hidden layers: 3
- Number of hidden neurons per layer: 20
- class column: Severity_bin (selected from a dropdown)
- ☐ Ignore Missing Values
- ☐ Use seed for random initialization
- Random seed: 982,163,972

A warning message at the bottom states: "Parameters 'hiddenlayer' and 'nrhiddenneurons' are controlled by variables." At the bottom of the dialog are buttons for "OK", "Apply", "Cancel", and a help icon.

After balancing and normalizing the training data, the MLP is trained using the RProp MLP Learner. The architecture of the network is provided through the hyperparameter optimization process, which tests 9 different configurations, each evaluated over 5-fold cross-validation inside the optimization loop. Once the best-performing architecture is identified, the final model is trained on the full balanced training set.

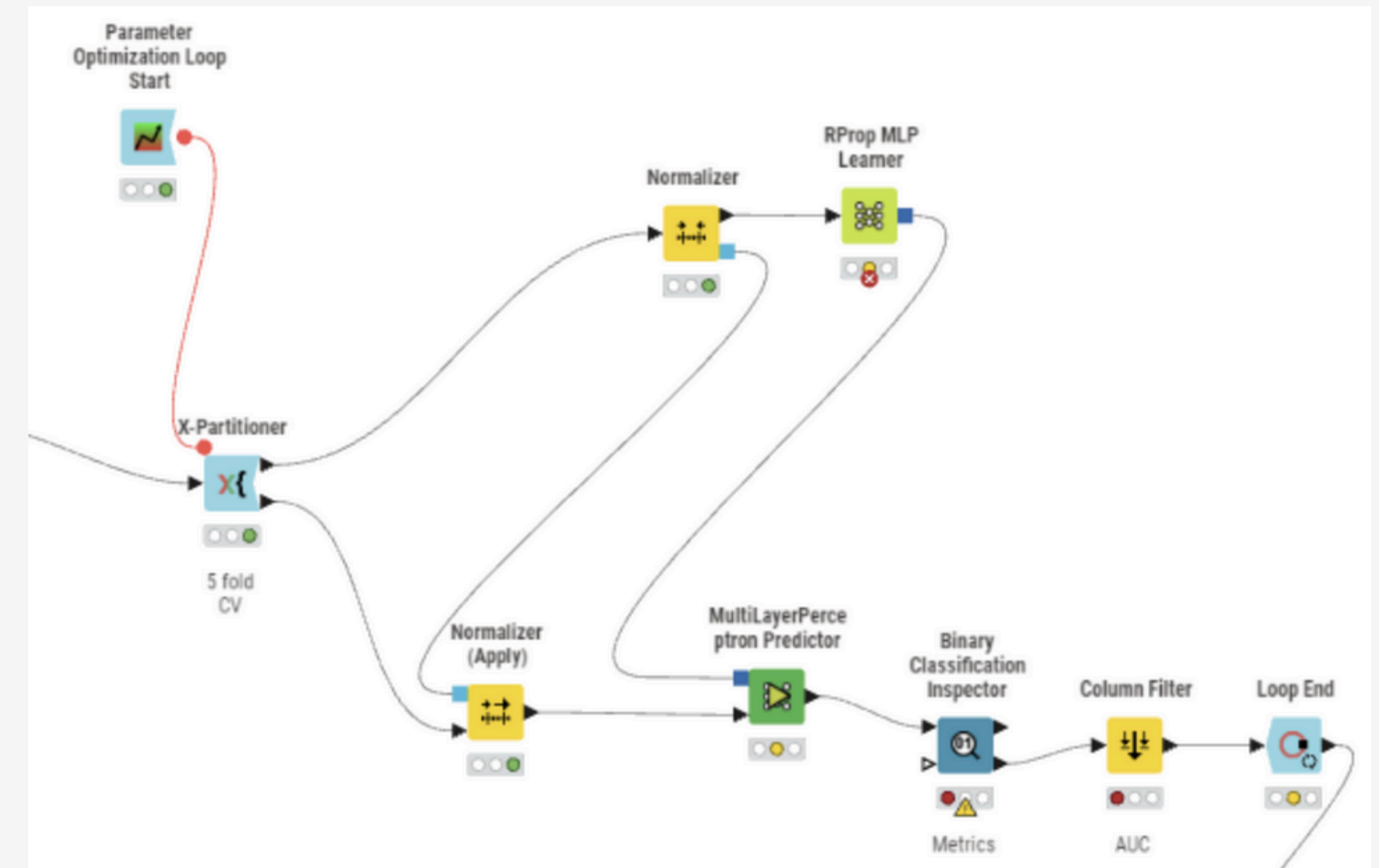
The MLP Predictor then applies this optimized model to the normalized 20% test set. This step produces the predicted classes and probabilities that are later evaluated in the performance section.

Hyperparameter Optimization

A dedicated hyperparameter optimization metanode is used to determine the best MLP architecture. The search space consists of 9 configurations, combining different numbers of hidden layers and neurons. These configurations are passed as flow variables to the RProp MLP Learner and evaluated through 5-fold cross-validation, resulting in 45 total training runs inside the metanode.

BALANCING INSIDE THE LOOP

To reduce computational load during repeated training, each cross-validation fold is balanced by downsampling both classes to 5,000 observations each, producing a training subset of 10,000 rows per fold. This ensures fair model comparisons while maintaining equal representation of severe and non-severe accidents during tuning.



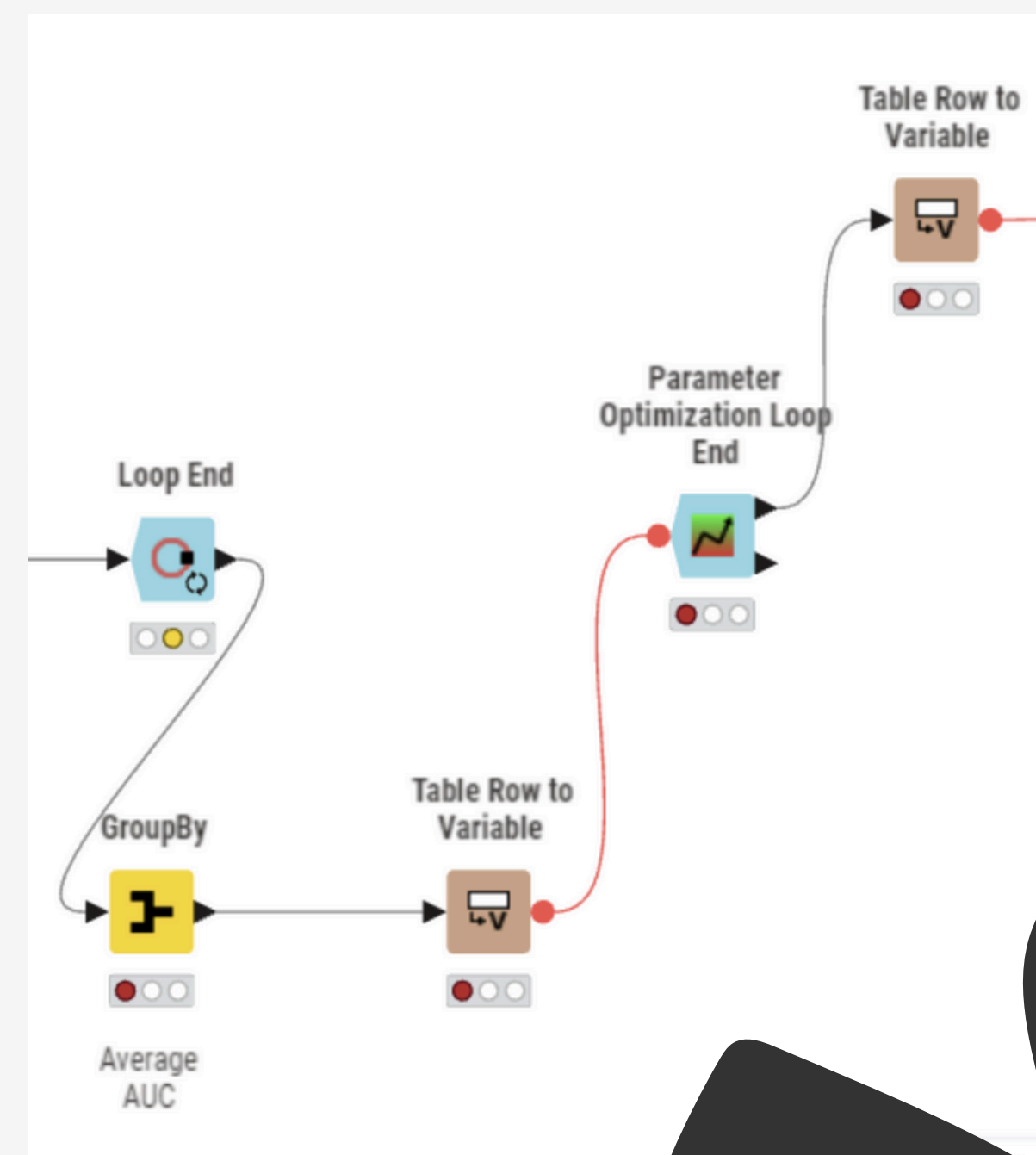
Hyperparameter Optimization

CROSS-VALIDATION PROCESS

For every architecture, the X-Partitioner generates five train/validation splits. Only the training portion of each fold is balanced and normalized. The validation portion remains untouched except for the application of the normalization model. This prevents data leakage and ensures that each architecture is evaluated under identical conditions.

MODEL SELECTION AND FINAL TRAINING

For each architecture, the AUC obtained from the five validation folds is averaged, and the Parameter Optimization Loop End selects the configuration with the highest mean AUC. This optimal architecture is then passed to the main workflow and retrained on the full balanced training set, ensuring the final model uses the structure validated through cross-validation while benefiting from all available training data.



Model Evaluation

BINARY CLASSIFICATION INSPECTION

The predictions generated by the MLP Predictor are evaluated using the Binary Classification Inspector, which provides the main performance metrics for the model. This includes accuracy, sensitivity, specificity, and the AUC based on the test set of approximately 100,000 observations. Since the test data is not involved in balancing, normalization, or hyperparameter tuning, these metrics offer an unbiased assessment of the MLP's performance and allow for a direct comparison with the other models implemented in this project.

The screenshot shows the 'Binary Classification Inspector (Metrics)' dialog box. The 'Main' tab is selected, showing various configuration options. The 'Maximum number of rows' is set to 30,000. The 'Target column' is 'Prediction (Severity_bin)'. The 'Binary threshold options' section shows a red error message: 'Column 'Prediction (Severity_bin)' contains no possible values'. The 'Model prediction columns' section has two panes: 'Exclude' and 'Include'. The 'Exclude' pane is active, showing a list of columns: WEB_AVG_ACCESS_LASTMONTH, WEB_WEEK_CHANGE_INDEX, WEB_N_COOKIE_LASTDAY, WEB_N_COOKIE_LASTFIVEDAYS, WEB_N_ACCESS_TOT, WEB_N_ACCESS_LASTDAY, and WEB_AVG_ACCESS_LASTWEEK. The 'Include' pane is empty. The 'Output data options' section has three checked options: 'Retain non-prediction columns in output table', 'Append new predictions for all models', and 'Append and label prediction column for the selected model'. The 'Initial threshold methods' section has 'Max Youden's Index' selected, and a threshold value of 0.5 is shown.

Dialog - 3:90:75 - Binary Classification Inspector (Metrics)

File

Confusion Matrix Main Prediction Distribution Style Interactivity Model Stats Chart Flow Variables Job Manager Selection Memory Policy ROC Curve

Maximum number of rows: 30,000

Target column: S Prediction (Severity_bin)

Binary threshold options

Positive class value: Column 'Prediction (Severity_bin)' contains no possible values

Model prediction columns

Manual Selection Wildcard/Regex Selection

Exclude

Filter

WEB_AVG_ACCESS_LASTMONTH
WEB_WEEK_CHANGE_INDEX
WEB_N_COOKIE_LASTDAY
WEB_N_COOKIE_LASTFIVEDAYS
WEB_N_ACCESS_TOT
WEB_N_ACCESS_LASTDAY
WEB_AVG_ACCESS_LASTWEEK

Enforce exclusion

Include

Filter

D P (Severity_bin=1)MLP

Enforce inclusion

Output data options

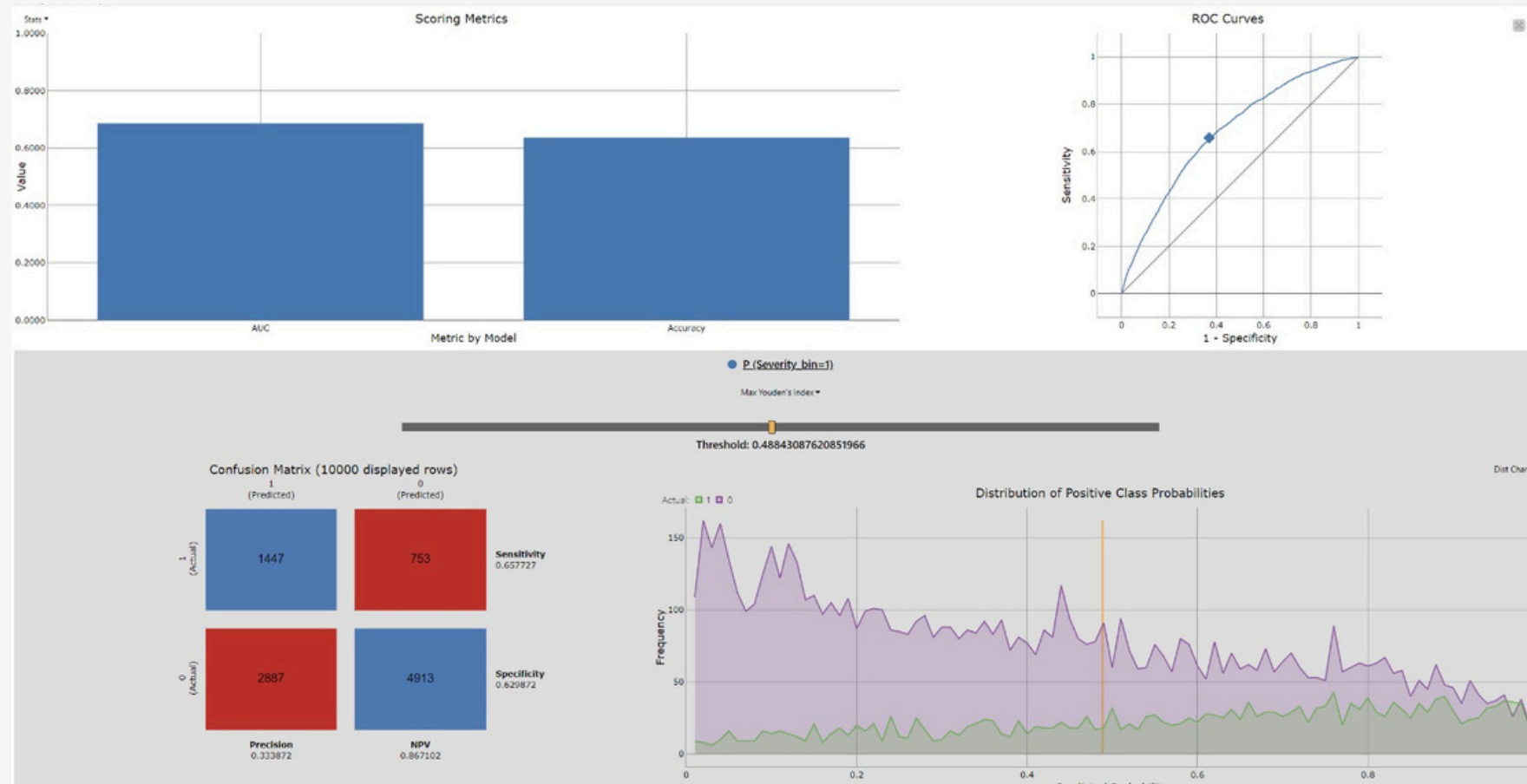
Retain non-prediction columns in output table
Append new predictions for all models
Append and label prediction column for the selected model

Label: Selected Model Predictions

Initial threshold methods

Max Youden's Index Max Sensitivity Max Specificity Max F-Measure
Max Accuracy Max Precision Custom 0.5

MLP Outcomes — Texas

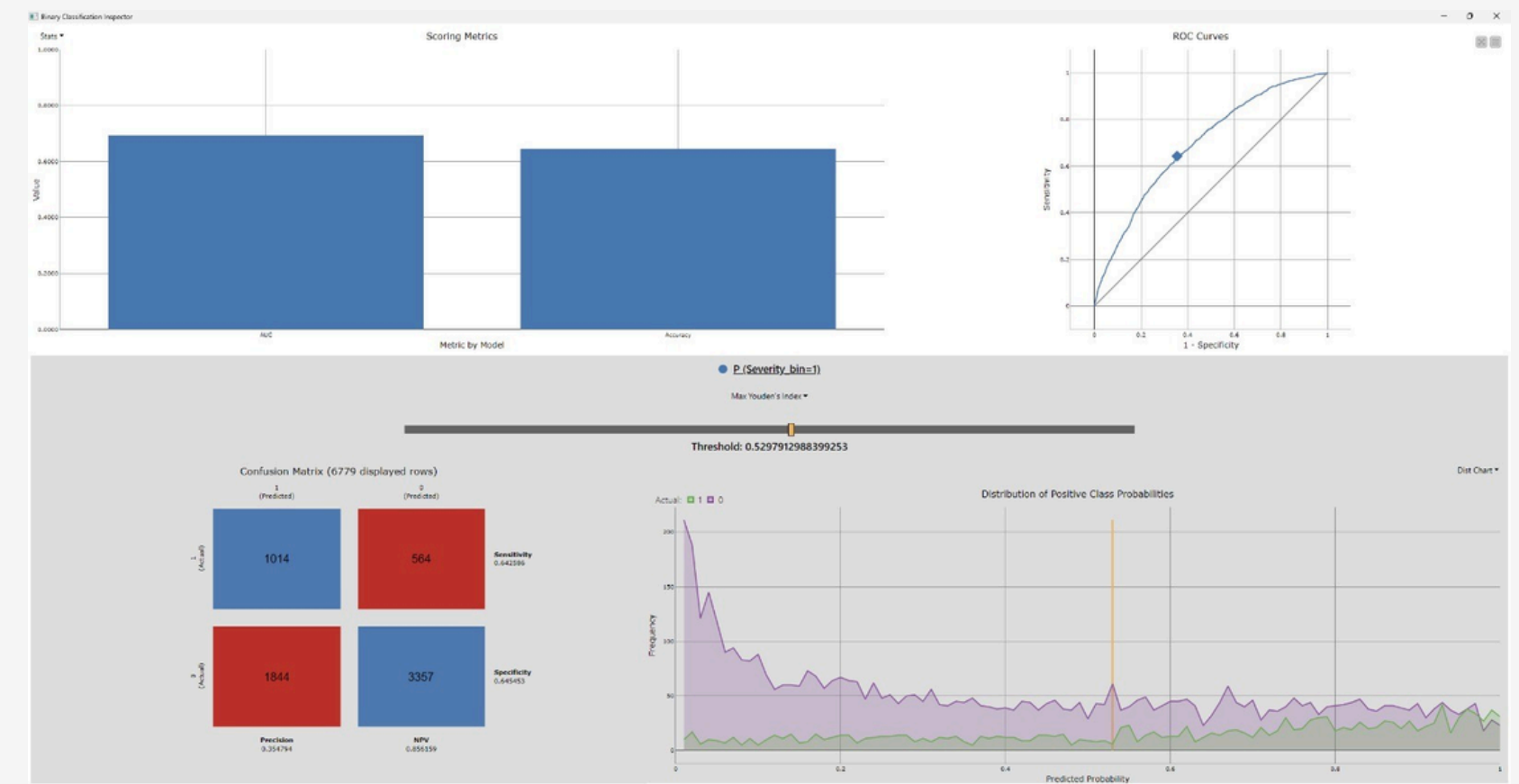


For Texas, the updated MLP results show an AUC of 0.686 and an accuracy of 0.636, indicating a moderate ability to distinguish between severe and non-severe accidents. Sensitivity reaches 0.6577 and specificity 0.6298, showing a reasonably balanced performance. The confusion matrix reports 1,447 true positives and 4,913 true negatives, alongside 753 false negatives and 2,887 false positives, meaning the model captures many severe cases but also tends to overpredict severity.

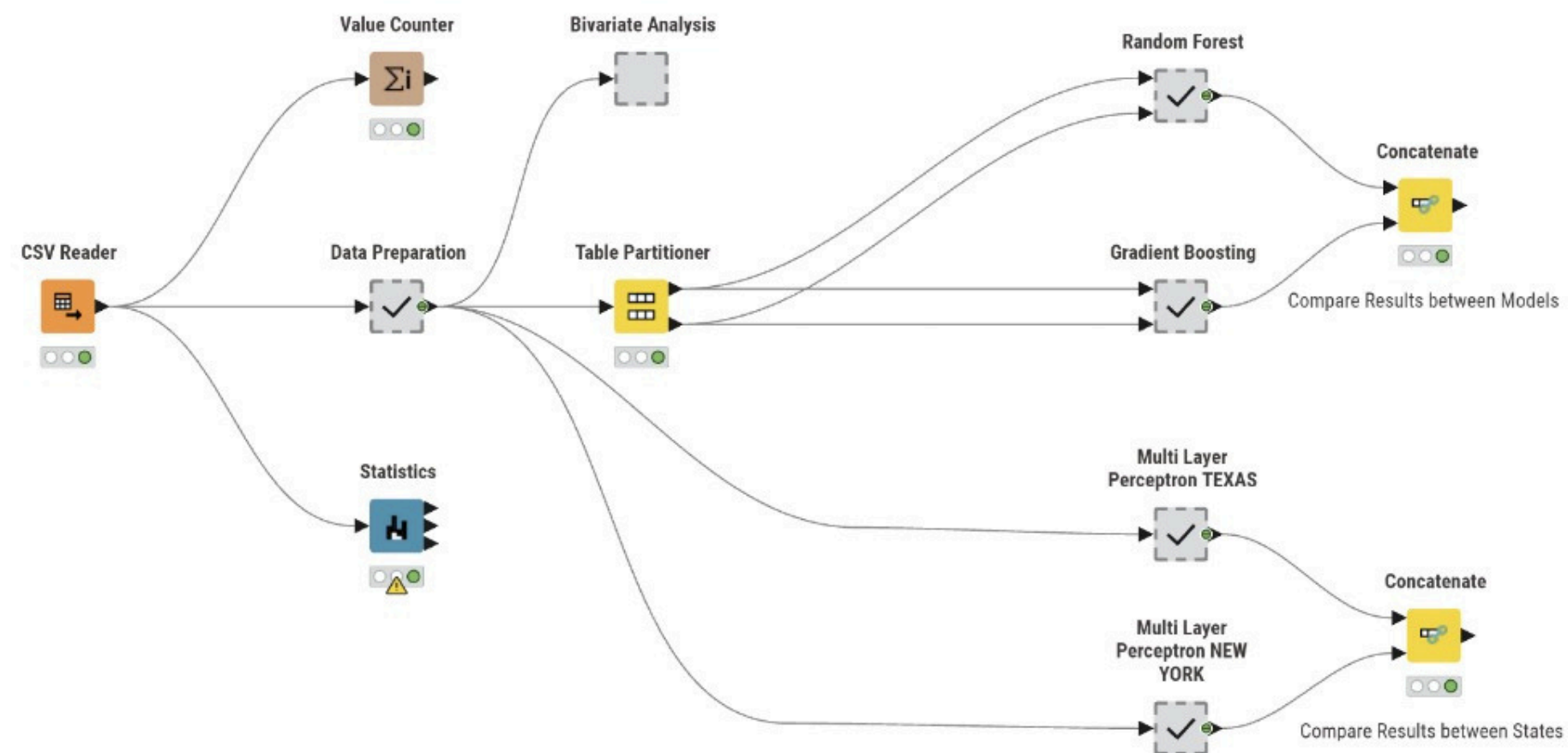
The probability distribution shows some separation between classes, and the optimal threshold of 0.4884 reflects the need to balance recall and precision. Overall, while performance is lower than earlier iterations, the model still leverages Texas's relatively uniform traffic structure—characterized by long highway segments and more predictable driving conditions—to extract meaningful severity patterns.

MLP Outcomes — New York

For New York, the MLP achieves an AUC of 0.693 and an accuracy of 0.645, indicating a moderate ability to distinguish between severe and non-severe accidents. Sensitivity reaches 0.6425, while specificity is 0.6455, showing that the model performs similarly across both classes. The confusion matrix reports 1,014 true positives and 3,357 true negatives, alongside 564 false negatives and 1,844 false positives, meaning the model captures many severe cases but also generates a notable number of false alarms.



The probability distribution shows limited class separation, and the optimal threshold of 0.5297 reflects the model's need to balance between recall and precision in a more variable traffic environment. Compared to Texas, New York's dense urban conditions—characterized by frequent intersections, irregular traffic flow, and rapidly changing road patterns—introduce more noise, making severity prediction more difficult for the MLP.



05

Outcomes & Comparisons

Random Forest vs XGBoost

To evaluate the performance of non-neural tree-based models, we compare the Simple Random Forest—the strongest RF variant in our workflow—with the Optimized XGBoost model. Both were trained on the full U.S. dataset after preprocessing and evaluated on the same 20% test set. This ensures that differences in performance reflect genuine modeling capabilities rather than data-related artifacts.

THE SIMPLE RANDOM FOREST

The Simple Random Forest achieves an AUC of 0.867 and an accuracy of 0.789, demonstrating strong discrimination between severe and non-severe accidents. RF benefits from aggregating many independently constructed trees, which reduces variance and provides stable predictions even in the presence of noisy or highly correlated features. The model performs particularly well at identifying non-severe accidents due to its balanced sensitivity–specificity tradeoff and the natural robustness of bagging methods to class imbalance. Its relatively high performance with minimal hyperparameter tuning also reflects the algorithm’s inherent ability to generalize across heterogeneous traffic conditions present in a national-scale dataset.

Metric	Random Forest	XGBoost
AUC	0,867	0,870
Accuracy	0,789	0,791
Sensitivity	0,790	0,799
Specificity	0,791	0,793
Threshold	default RF	0,169

Random Forest vs XGBoost

THE OPTIMIZED XGBOOST

The XGBoost model, tuned through hyperparameter optimization and evaluated at its optimal threshold (0.169), achieves slightly higher metrics with an AUC of 0.870 and accuracy of 0.791. This improvement, although modest, is consistent with the theoretical advantages of gradient boosting, which incrementally builds trees that correct the errors of prior iterations. As a result, XGBoost is more capable than Random Forest at capturing subtle nonlinear interactions—such as the interplay between weather variables, temporal patterns, and infrastructure characteristics—that affect accident severity. The ROC curve displays a marginally sharper bend in the upper-left region, indicating improved sensitivity at comparable levels of specificity.

Furthermore, XGBoost tends to optimize classification boundaries more aggressively, which helps it identify additional severe cases without a significant increase in false positives. The tuned threshold (0.169) reflects this behavior, shifting the decision boundary to prioritize early detection of severe accidents.

OVERALL COMPARISON

While both models perform exceptionally well, XGBoost demonstrates a small but consistent performance advantage. This edge stems from its ability to model complex residual patterns and interactions that Random Forest may overlook due to its independent-tree construction. However, the gap between the two models is narrow, suggesting that accident severity is influenced by strong, consistently informative predictors that both algorithms can capture effectively.

These results set the stage for a deeper comparison using feature importance metrics, which help explain how each algorithm prioritizes inputs and why their performance levels are so similar. The next section examines these interpretability differences in detail.

Random Forest vs XGBoost

Feature Importance

Both models identify broadly similar drivers of accident severity, but they differ in how they rank and distribute importance due to their underlying learning mechanisms. Random Forest measures importance based on how frequently a feature is used for data splits across many trees, whereas XGBoost measures how much each feature contributes to reducing error across boosted iterations.

RANDOM FOREST KEY FEATURES

The most influential variables for the Simple Random Forest model are:

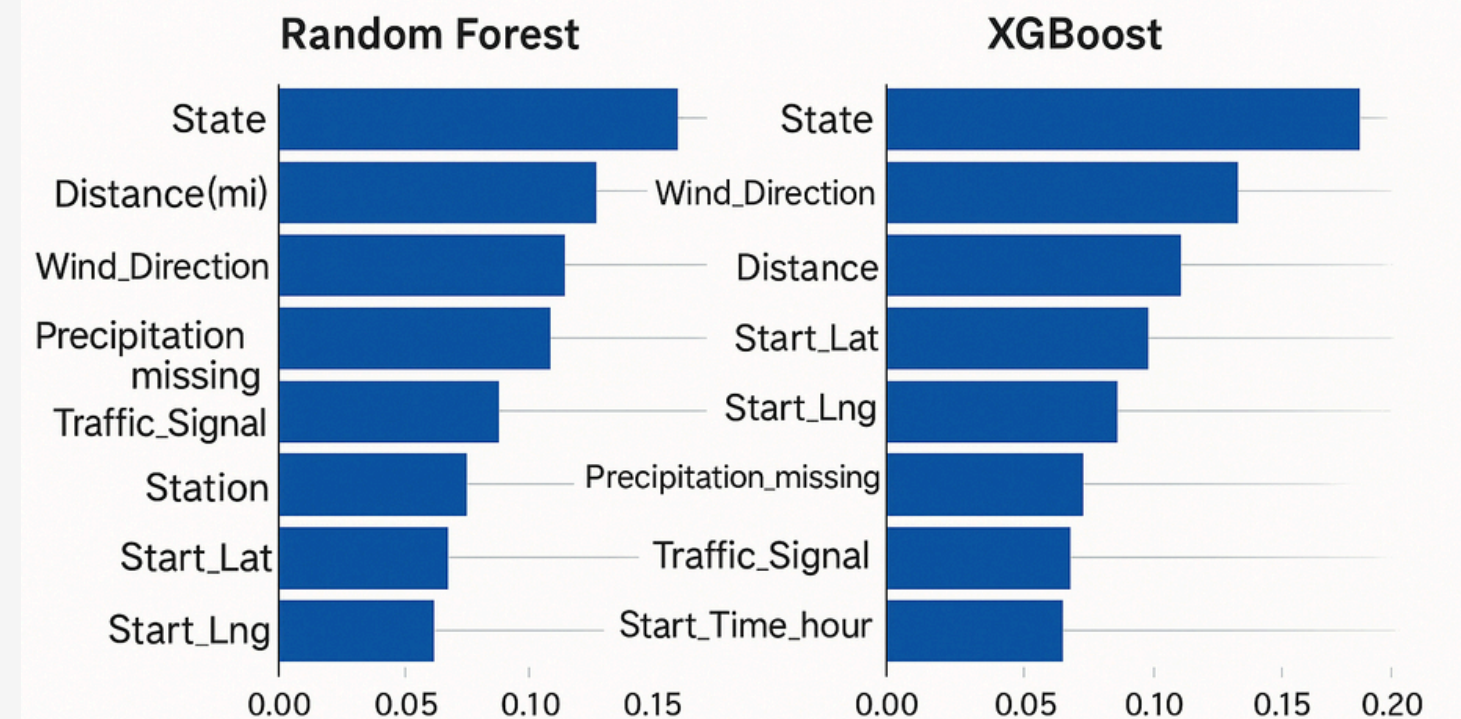
State, Distance(mi), Wind_Direction, Precipitation_missing, Traffic_Signal, Station, Start_Lat and Start_Lng, Visibility, Humidity, Pressure, Start_Time_hour, Start_Time_dayOfWeek

Interpretation:

Random Forest prioritizes variables that repeatedly create strong, early splits across many independent trees.

Because of this: State dominates importance, as it consistently provides the highest-quality splits across the ensemble. Distance, Wind Direction, and Traffic_Signal reliably partition the dataset and therefore score highly. Importance is spread across many features, with geography and basic road/weather indicators leading.

Variable Importance Comparison



Random Forest vs XGBoost

Feature Importance

XGBOOST KEY FEATURES

According to XGBoost's scaled importance, the top predictors are: State, Wind_Direction, Distance(mi), Start_Lat, Start_Lng, Precipitation_missing, Traffic_Signal, Start_Time_hour, Weather & atmospheric variables.

Interpretation:

XGBoost focuses on features that consistently reduce loss across sequential trees. It:

1. Preserves the dominance of State,
2. Gives stronger weight to continuous predictors such as latitude, longitude, humidity, and temperature,
3. Captures interaction effects (e.g., weather $\times$ geography $\times$ time) that Random Forest may dilute.

OVERALL INTERPRETATION

Both models overwhelmingly agree that State is the strongest predictor of accident severity. Both highlight Distance, Wind Direction, and multiple weather-related variables as critical determinants. XGBoost spreads importance more granularly across continuous variables and captures subtle nonlinear interactions. Random Forest concentrates importance on features that consistently produce stable splits across the ensemble. The consistency across models strongly reinforces the conclusion that accident severity is shaped by geographic conditions, road infrastructure, and regional weather patterns. This convergence across two distinct algorithms provides robust evidence supporting the state-level heterogeneity highlighted earlier and strengthens the managerial implications presented later in the report.

MLP Analysis: Texas vs New York

The updated state-level evaluation of the MLP continues to highlight meaningful differences in how accident severity behaves across regions, even though both states now show moderately similar performance levels. Despite using the same modeling workflow, the MLP responds differently to each state’s traffic environment, reflecting the structural conditions governing accident patterns.

TEXAS

In Texas, the MLP achieves an AUC of 0.686 and an accuracy of 0.636, indicating a moderate but useful ability to distinguish between severe and non-severe accidents. Sensitivity reaches 0.6577, while specificity is 0.6298, showing relatively balanced performance across both classes. The confusion matrix reports 1,447 true positives but also 753 false negatives and 2,887 false positives, suggesting that the model successfully captures many severe events but tends to overpredict severity. The probability distribution shows some separation between classes, and the optimal threshold of 0.4884 indicates that a lower decision boundary improves recall. These results remain consistent with Texas’s road network characteristics, where long highway segments and more stable traffic flows enable the MLP to detect severity patterns with reasonable reliability.

MLP State-Level Performance Comparison		
Metric	Texas	New York
AUC	0,686	0,693
Accuracy	0,636	0,645
Sensitivity (Recall)	0,6577	0,6425
Specificity	0,6298	0,6455
Precision	0,3339	0,3549
NPV	0,8671	0,8560
Optimal Threshold	0,4884	0,5297
True Positives (TP)	1,447	1,014
False Negatives (FN)	753	564
False Positives (FP)	2,887	1,844
True Negatives (TN)	4,913	3,357

MLP Analysis: Texas vs New York

NEW YORK

In New York, performance metrics improve slightly relative to Texas, with an AUC of 0.693 and an accuracy of 0.645. Sensitivity (0.6425) and specificity (0.6455) are nearly identical, demonstrating a balanced classification outcome. The confusion matrix shows 1,014 true positives and 3,357 true negatives, alongside 564 false negatives and 1,844 false positives. The probability distribution, however, remains flatter with greater overlap between classes, and the optimal threshold of 0.5297 is higher than in Texas. This underscores the complexity of New York’s urban environment—dense intersections, variable traffic flow, and rapidly changing driving conditions introduce noise that reduces class separability.

COMPARISON

While the updated metrics show Texas and New York performing at similar accuracy and AUC levels, the underlying patterns differ. Texas benefits from more uniform and predictable road conditions, resulting in clearer severity signals and a lower optimal threshold. New York, despite slightly higher AUC and accuracy, shows greater distributional overlap and requires a higher threshold due to its complex urban dynamics. These results confirm that severity prediction remains state-dependent, shaped by infrastructure, congestion patterns, and local driving behavior. They also reinforce the methodological decision to train MLP models at the state level, as a single national model would struggle to capture such variability.

MLP State-Level Performance Comparison		
Metric	Texas	New York
AUC	0,686	0,693
Accuracy	0,636	0,645
Sensitivity (Recall)	0,6577	0,6425
Specificity	0,6298	0,6455
Precision	0,3339	0,3549
NPV	0,8671	0,8560
Optimal Threshold	0,4884	0,5297
True Positives (TP)	1,447	1,014
False Negatives (FN)	753	564
False Positives (FP)	2,887	1,844
True Negatives (TN)	4,913	3,357

06

Managerial Implications

Managerial Implications & Conclusions

Our analysis reveals a consistent and compelling message across all models: accident severity in the United States is predominantly shaped by geographic and infrastructural differences across states, rather than by isolated crash-level characteristics alone. This finding has direct implications for policy-makers, transportation authorities, emergency services, and insurance companies seeking to allocate resources effectively and design targeted interventions.

State-Level Heterogeneity as the Primary Driver of Severity

In every model tested—Random Forest, Gradient Boosting, and XGBoost—State emerges as the single most influential predictor. This aligns with the observation that severity patterns differ widely across the country due to variation in road design, speed regimes, weather systems, enforcement intensity, and emergency response capacity. As shown in the feature importance results (page 62), even models that account for dozens of environmental and temporal factors consistently rank State above all others.

This implies that national-level strategies are fundamentally limited. Instead, transportation authorities should:

- Identify high-severity states and prioritize investments in those regions.
- Use low-severity states as benchmarks, replicating effective design standards, legislation, and response protocols where applicable.
- Develop state-specific safety policies, rather than applying uniform interventions nationwide.
- Incorporate geographic risk profiles into insurance pricing and public budgeting.

States differ not just in weather or infrastructure but in the systemic environment in which accidents occur, and policies must reflect this heterogeneity.

Managerial Implications & Conclusions

Interpretation of Highly Ranked Variables: Structural Proxies, Not Direct Causes

Several variables that appear important—most notably Wind_Direction and Distance(mi)—function as proxies for deeper structural conditions rather than causal factors. The importance of these variables should therefore be interpreted carefully.

- Wind Direction correlates strongly with regional weather systems, storm patterns, and seasonal climatic phenomena (e.g., Gulf Coast hurricanes, Pacific moisture, Plains crosswinds). Its importance reflects the geographic variability in hazardous weather, not the directional vector itself.
- Distance (road segment length) is closely tied to highway type and infrastructure design. Long segments typically occur on high-speed corridors where severe accidents have a greater operational impact. The model assigns high importance to Distance because it indirectly encodes road class, speed regime, and congestion propagation, not because long segments inherently cause severity.

For managers, this means interventions should target the underlying structural conditions these variables represent—high-speed road sections, vulnerable climates, and infrastructure bottlenecks—rather than the variables themselves.

What the MLP Confirms: The Drivers of Severity Change Across States

To test whether these geographic effects were artifacts of tree-based models, we developed state-specific MLPs. The results for Texas and New York (pages 56–57) show that severity patterns depend strongly on the traffic environment:

- Texas shows clearer class separation and higher recall: severity follows predictable highway-dominated patterns, allowing for earlier incident detection and smoother model performance.
- New York shows flatter probability distributions and higher thresholds: dense intersections, inconsistent traffic flow, and complex road geometries create a noisier, more unpredictable environment.

This confirms that severity prediction is inherently state-specific, validating the managerial need for tailored models and policies.

Managerial Implications & Conclusions

Practical Recommendations for Transportation Agencies

Based on the technical findings, actionable managerial steps include:

For predictable, highway-dominant states (e.g., Texas):

- Deploy real-time severity prediction tools to assist with dynamic rerouting, queue management, and proactive emergency resource allocation.
- Integrate model outputs into operational dashboards (tow-truck positioning, lane-closure planning).
- Invest in targeted improvements for long, high-speed corridors where severity risk is structurally elevated.

For dense, heterogeneous states (e.g., New York):

- Complement predictive models with richer data streams (real-time congestion, connected-vehicle telemetry, intersection-level road attributes).
- Implement hybrid decision-support systems combining model predictions with operator expertise, due to higher false-negative risks.
- Prioritize infrastructure modernization in complex urban areas—improved signaling, lighting, and intersection redesign.

Validation Across Models: A Robust Set of Insights

The consistency observed across the Basic Random Forest, Gradient Boosting, XGBoost, and MLP strengthens the reliability of these managerial insights. Despite their architectural differences, all models converge on the same message:

- Severity is regional, structural, and context-dependent.
- Policies must reflect localized risk, not national averages.
- The most influential features are system-level indicators (state, road type, geography, climate) rather than micro-level crash details.

This alignment across models provides high confidence that the managerial implications outlined above are both reliable and actionable.

Managerial Implications & Conclusions

Final Managerial Conclusion

Accident severity prediction cannot be effectively addressed with a one-size-fits-all approach. States with stable, predictable traffic patterns can benefit from automation and early-warning systems, while states characterized by dense and complex roadway networks require richer data, specialized models, and tailored policy interventions.

By leveraging the insights from this analysis, transportation agencies can optimize safety investments, enhance emergency response, and reduce congestion, ultimately improving mobility and road safety across varying U.S. environments.